

프로세스·스레드·동시성·동기화

실행 모델과 병렬성에서 Mutex·Condition Variable까지

01 프로세스와 스레드

02 동시성과 병렬성

03 Mutex와 Condition Variable

원본 Markdown 9개의 내용·코드·표·수식을 보존하여 연관 주제별로 편집

title: 프로세스와 스레드 aliases:

tags:

created: 2026-08-01 updated: 2026-08-01

- Process and Thread
- CS
- OS
- Process
- Thread
- Cpp

프로세스와 스레드

[!summary] 한 문장 정리
프로세스는 자원 소유와 보호의 단위이고, 스레드는 프로세스 내부에서 CPU 명령어를 실행하는 흐름이다.

학습 목표

- 프로그램, 프로세스, 스레드의 차이를 설명한다.
- 프로세스의 상태 전이와 문맥 교환을 이해한다.
- 프로세스와 스레드가 어떤 자원을 공유하고 분리하는지 이해한다.
- CPU 스케줄링의 목적과 대표 알고리즘을 구분한다.
- 경쟁 조건, 임계 영역, mutex, 조건 변수, 세마포어를 이해한다.
- 교착 상태의 발생 조건과 방지 방법을 설명한다.
- C++에서 프로세스와 스레드를 안전하게 사용한다.

참고 자료

- [C:\Users\user\Desktop\OS\ocs.pdf](#)
- 프로세스, 스레드와 병행성, CPU 스케줄링, 동기화 도구와 사례, 교착 상태
- [C:\Users\user\Desktop\OS\ostep.pdf](#)
- 프로세스 가상화, 프로세스 API, 제한적 직접 실행, 스케줄링, 병행성

이 노트의 범위	ocs.pdf 대응 내용	ostep.pdf 대응 내용
프로세스와 IPC	프로세스 장	프로세스 개념, 프로세스 API
실행 제어와 스케줄링	CPU 스케줄링 장	제한적 직접 실행, 스케줄링 장들
스레드	스레드와 병행성 장	병행성 개요, 스레드 API
동기화	동기화 도구와 동기화 사례 장	락, 조건 변수, 세마포어
오류와 교착 상태	교착 상태 장	병행성 관련 오류 장

[!note] 2차 검증

이 문서는 2026-08-01에 위 두 PDF의 관련 장을 다시 대조하고 C++ 예제를 재검토하여 보완한 최종본이다.

1. 프로그램, 프로세스, 스레드

1.1 프로그램

프로그램(program)은 디스크에 저장된 실행 파일과 정적 데이터의 묶음이다. 아직 실행되지 않은 수동적인 존재다.

1.2 프로세스

프로세스(process)는 **실행 중인 프로그램**이다. 실행 코드뿐 아니라 운영체제가 프로그램을 실행하기 위해 관리하는 환경 전체를 포함한다.

- 가상 주소 공간
- 코드와 전역 데이터
- 힙(heap)
- 하나 이상의 스택(stack)
- CPU 레지스터와 프로그램 카운터
- 프로세스 ID
- 열린 파일과 소켓
- 보안 권한
- 스케줄링 정보

같은 프로그램을 두 번 실행하더라도 서로 다른 프로세스가 생성된다. 두 프로세스는 같은 실행 파일을 사용하지만 서로 다른 주소 공간과 실행 상태를 갖는다.

1.3 스레드

스레드(thread)는 프로세스 안에서 실제로 명령어를 실행하는 흐름이다. 하나의 프로세스는 하나 이상의 스레드를 가질 수 있다.

같은 프로세스의 스레드들이 공유하는 것:

- 코드
- 전역 변수와 정적 변수
- 힙
- 열린 파일과 소켓
- 프로세스의 주소 공간

각 스레드가 별도로 갖는 것:

- 프로그램 카운터
- CPU 레지스터
- 스택
- 스레드 ID
- 스레드 지역 저장소
- 실행 및 스케줄링 상태

1.4 프로세스와 스레드 비교

구분	프로세스 사이	같은 프로세스의 스레드 사이
가상 주소 공간	기본적으로 분리하되 공유 메모리 매핑 가능	공유
전역 변수와 힙	분리	공유
스택	프로세스별 분리	스레드별 분리
열린 파일	기본적으로 별도 관리하나 <code>fork()</code> 시 파일 디스크립터가 상속될 수 있음	프로세스의 파일 디스크립터 테이블 공유
데이터 교환	IPC 필요	공유 메모리로 간단
생성 및 전환 비용	상대적으로 큼	상대적으로 작음
장애 격리	좋음	나쁨
주요 위험	IPC 복잡성	데이터 경쟁과 교착 상태

[!important]

현대 운영체제에서 프로세스는 주로 **자원과 보호의 단위**, 커널 스레드는 **CPU 스케줄링의 단위**로 사용된다.

[!warning] 파일 공유의 함정

같은 프로세스의 스레드는 파일 디스크립터를 공유한다. 여러 스레드가 같은 파일 객체의 현재 오프셋을 동시에 변경하면 예상하지 못한 위치를 읽거나 쓸 수 있으므로 별도의 동기화 또는 위치 기반 I/O가 필요할 수 있다.

2. 병행성과 병렬성

2.1 병행성

병행성(concurrency)은 여러 작업의 실행 시간이 겹치는 성질이다. CPU가 하나여도 운영체제가 작업을 빠르게 번갈아 실행하면 병행성이 생긴다.

TEXT

시간 —————>
CPU [스레드 A][스레드 B][스레드 A][스레드 B]

2.2 병렬성

병렬성(parallelism)은 여러 CPU 코어에서 실제로 작업이 동시에 실행되는 것이다.

TEXT

CPU 0 [———— 스레드 A —————]
CPU 1 [———— 스레드 B —————]

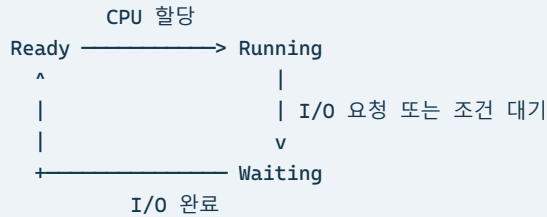
멀티스레드 프로그램이라고 반드시 병렬 실행되는 것은 아니다. 실제 병렬 실행 여부는 CPU 코어 수, 운영체제 스케줄링, 다른 작업의 부하에 따라 달라진다.

3. 프로세스 상태

대표적인 프로세스 상태는 다음과 같다.

- **New**: 프로세스가 생성되는 중
- **Ready**: CPU가 할당되면 실행할 수 있는 상태
- **Running**: CPU에서 실행 중
- **Waiting/Blocked**: I/O 완료나 특정 사건을 기다리는 상태
- **Terminated**: 실행이 끝난 상태

TEXT



Running — 선점 또는 타임 슬라이스 만료 —> Ready
 Running — 정상 종료 또는 오류 —> Terminated

3.1 Ready와 Waiting의 차이

- Ready 상태는 CPU만 받으면 즉시 실행할 수 있다.
- Waiting 상태는 CPU를 받아도 실행할 수 없다. I/O 완료나 특정 조건 충족이 먼저 필요하다.

3.2 스케줄링 큐

- ready queue에는 CPU를 기다리는 실행 가능한 작업이 들어간다.
- I/O wait queue에는 특정 장치나 사건의 완료를 기다리는 작업이 들어간다.
- I/O가 완료되면 작업은 Waiting에서 Ready로 이동할 뿐 곧바로 실행된다고 보장되지는 않는다.
- 각 큐에서 다음 작업을 선택하는 기준이 스케줄링 정책이다.

4. PCB와 문맥 교환

4.1 PCB

운영체제는 각 프로세스의 정보를 PCB(Process Control Block)에 저장한다.

- 현재 상태
- 프로그램 카운터
- CPU 레지스터
- 스케줄링 우선순위

- 메모리 관리 정보
- 열린 파일 정보
- CPU 사용량
- 부모와 자식 관계

스레드의 실행 문맥은 TCB(Thread Control Block) 또는 운영체제의 태스크 구조체에 저장할 수 있다.

4.2 문맥 교환

문맥 교환(context switch)은 현재 실행 중인 스레드의 실행 문맥을 저장하고 다른 스레드의 문맥을 복원하는 과정이다.

1. 현재 프로그램 카운터와 레지스터를 저장한다.
2. 현재 스레드의 상태를 변경한다.
3. 스케줄러가 다음 스레드를 선택한다.
4. 선택된 스레드의 레지스터와 실행 위치를 복원한다.
5. 해당 스레드의 실행을 재개한다.

문맥 교환은 응용 프로그램의 작업을 직접 진행하지 않는 오버헤드다. 프로세스 사이의 전환은 주소 공간, 페이지 테이블, TLB 및 캐시에 영향을 주므로 같은 프로세스 내 스레드 전환보다 일반적으로 무겁다.

[!warning]

스레드 전환도 무료가 아니다. 캐시 손실, 커널 진입, 코어 이동, 락 경합으로 비용이 커질 수 있다.

5. 제한적 직접 실행

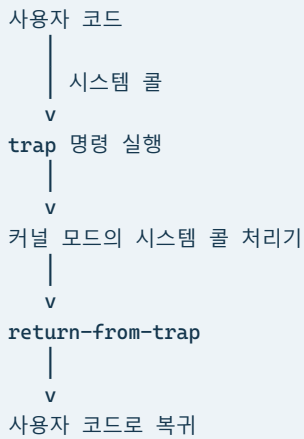
운영체제는 성능을 위해 사용자 프로그램의 명령어를 CPU에서 직접 실행한다. 하지만 프로그램에 CPU 전체 권한을 주면 시스템 보호와 제어가 불가능하다.

이를 해결하는 핵심 장치는 다음과 같다.

- 사용자 모드와 커널 모드
- 시스템 콜
- trap 명령
- 인터럽트
- 타이머 인터럽트

5.1 시스템 콜

프로그램이 파일 읽기, 프로세스 생성, 네트워크 통신 같은 특권 작업을 요청하면 시스템 콜을 사용한다.

TEXT

5.2 타이머 인터럽트

프로세스가 자발적으로 CPU를 반환하지 않더라도 타이머 인터럽트가 발생하면 운영체제가 다시 제어권을 얻는다. 이를 통해 선점형 멀티태스킹이 가능해진다.

6. 프로세스 생성과 종료

Unix 계열 운영체제는 다음 API를 중심으로 프로세스를 관리한다.

- `fork()` : 현재 프로세스를 복제해 자식 프로세스를 생성한다.
- `exec()` : 현재 프로세스의 프로그램과 주소 공간을 새 프로그램으로 교체한다.
- `wait()` / `waitpid()` : 부모가 자식의 종료를 기다리고 종료 상태를 회수한다.

`fork()` 이후에는 부모와 자식이 모두 다음 명령어부터 실행한다. 어느 프로세스가 먼저 실행될지는 운영체제 스케줄러가 결정한다.

6.1 C++와 POSIX 프로세스 예제

[!note]

C++ 표준 라이브러리에는 범용적인 프로세스 생성 기능이 없다. 다음 코드는 Linux, macOS 또는 WSL에서 사용하는 POSIX 예제다.

CPP

```

#include <sys/types.h>
#include <sys/wait.h>
#include <unistd.h>

#include <cstring>
#include <iostream>

int main() {
    int fd[2];

    if (pipe(fd) == -1) {
        std::cerr << "pipe failed\n";
        return 1;
    }

    pid_t pid = fork();

    if (pid == -1) {
        std::cerr << "fork failed\n";
        return 1;
    }

    if (pid == 0) {
        close(fd[0]);

        const char* message = "message from child";
        write(fd[1], message, std::strlen(message));

        close(fd[1]);
        _exit(0);
    }

    close(fd[1]);

    char buffer[128]{};
    ssize_t size = read(fd[0], buffer, sizeof(buffer) - 1);

    if (size > 0) {
        std::cout << "parent received: " << buffer << '\n';
    }

    close(fd[0]);

    int status = 0;
    waitpid(pid, &status, 0);

    if (WIFEXITED(status)) {
        std::cout << "child exit status: "
                  << WEXITSTATUS(status) << '\n';
    }
}

```

BASH

```
g++ -std=c++20 -O2 process.cpp -o process
```

Windows에서는 `fork()` 대신 `CreateProcessW()` , 파이프에는 `CreatePipe()` 를 사용한다.

6.2 종료 후의 상태

- Unix에서 자식이 종료했지만 부모가 아직 `wait()` 로 종료 상태를 회수하지 않으면 zombie가 된다.
- 부모가 먼저 종료한 자식 프로세스는 orphan이 되며 운영체제의 지정된 프로세스가 부모 역할을 넘겨받아 종료 상태를 회수한다.
- 부모는 자식의 종료 상태를 회수해 프로세스 테이블의 항목이 정리되도록 해야 한다.

7. 프로세스 간 통신

분리된 주소 공간을 가진 프로세스끼리 데이터를 교환하려면 IPC(Inter-Process Communication)가 필요하다.

7.1 공유 메모리

여러 프로세스가 동일한 물리 메모리를 각자의 주소 공간에 매핑한다.

장점:

- 대량 데이터 전달이 빠르다.
- 반복적인 커널 복사 비용을 줄일 수 있다.

단점:

- 별도의 동기화가 필요하다.
- 경쟁 조건과 데이터 손상이 발생할 수 있다.

7.2 메시지 전달

프로세스가 `send()` 와 `receive()` 형태로 메시지를 주고받는다.

- 익명 파이프
- 이름 있는 파이프
- 메시지 큐
- Unix 도메인 소켓
- TCP/UDP 소켓

상황	적합한 방식
부모와 자식 사이의 단순 통신	파이프
같은 컴퓨터의 독립 서비스	Unix 도메인 소켓 또는 named pipe
원격 컴퓨터와 통신	TCP/UDP
매우 큰 데이터 공유	공유 메모리 + 동기화 도구

7.3 메시지 전달의 설계 선택

- **직접 통신:** 송신자와 수신자가 상대 프로세스를 명시한다.
- **간접 통신:** mailbox 또는 port를 통해 메시지를 전달한다.
- **blocking send/receive:** 작업이 완료되거나 메시지가 도착할 때까지 호출자가 기다린다.
- **non-blocking send/receive:** 즉시 반환하고 완료 여부를 나중에 확인한다.

- **무용량 버퍼**: 송신자와 수신자가 직접 rendezvous해야 한다.
- **유한 버퍼**: 큐가 가득 차면 송신자가 대기하거나 실패한다.
- **무한 버퍼라는 추상화**: 송신자가 대기하지 않는 것처럼 보이지만 실제 메모리는 유한하므로 역압력과 오류 정책이 필요하다.

7.4 클라이언트-서버 통신

- 소켓은 같은 컴퓨터 또는 네트워크의 프로세스를 연결한다.
- RPC는 원격 요청을 지역 함수 호출처럼 표현하지만 직렬화, 부분 실패, 재시도, 중복 실행을 고려해야 한다.
- 파이프는 바이트 스트림이므로 메시지 경계를 애플리케이션이 정의해야 할 수 있다.
- 공유 메모리는 빠르지만 프로세스 충돌 격리와 동기화 설계가 더 복잡하다.

8. 멀티스레딩의 장점과 위험

8.1 장점

- **응답성**: 한 작업이 기다리는 동안 UI나 다른 요청을 처리한다.
- **자원 공유**: 같은 주소 공간을 사용하므로 데이터 교환이 간단하다.
- **경제성**: 프로세스보다 생성과 전환 비용이 작은 편이다.
- **확장성**: 여러 CPU 코어를 활용할 수 있다.
- **I/O 중첩**: 한 스레드가 I/O를 기다릴 때 다른 스레드가 실행된다.

8.2 위험

- 데이터 경쟁
- 원자성 위반
- 실행 순서 위반
- 교착 상태
- 기아
- 라이브락
- 스레드 과다 생성
- 캐시 경합과 false sharing

9. 사용자 스레드와 커널 스레드

9.1 사용자 스레드

사용자 스레드(user thread)는 사용자 공간의 스레드 라이브러리가 생성과 전환을 관리한다.

- 커널 진입 없이 전환할 수 있어 빠를 수 있다.
- 운영체제가 각 사용자 스레드를 직접 알지 못할 수 있다.
- 구현 모델에 따라 한 스레드의 블로킹 시스템 콜이 프로세스 전체를 멈출 수 있다.

9.2 커널 스레드

커널 스레드(kernel thread)는 운영체제가 직접 관리하고 스케줄링한다.

- 한 스레드가 블로킹되어도 같은 프로세스의 다른 스레드를 실행할 수 있다.
- 여러 CPU 코어에서 실제 병렬 실행할 수 있다.
- 생성과 전환에 커널의 관리 비용이 든다.

9.3 매핑 모델

모델	구조	장점	단점
다대일	여러 사용자 스레드 → 하나의 커널 스레드	사용자 공간에서 관리가 빠름	한 스레드가 블로킹되면 전체가 멈출 수 있고 멀티코어 병렬성이 제한됨
일대일	사용자 스레드 하나 → 커널 스레드 하나	블로킹 격리와 멀티코어 병렬 실행	스레드 수가 많으면 커널 관리 비용 증가
다대다	여러 사용자 스레드 → 여러 커널 스레드	병렬성과 사용자 수준 관리의 유연성	런타임과 운영체제 구현이 복잡
2단계	다대다 + 일부 사용자/커널 스레드 고정	특정 작업에 커널 실행 단위를 보장	구현 복잡성 증가

현대 데스크톱 운영체제에서 `std::thread` 는 일반적으로 네이티브 커널 스레드와 연결된다. 다만 정확한 구현 세부 사항은 C++ 표준이 아니라 표준 라이브러리와 운영체제가 결정한다.

10. 스레드 생명주기와 암시적 스레딩

10.1 생성, 합류, 분리

- 생성: 새 실행 흐름과 스택을 준비한다.
- `join()` : 다른 스레드의 종료를 기다리고 실행 결과의 가시성을 확보한다.
- `detach()` : 스레드를 독립 실행시키지만 수명과 자원 관리가 어려워진다.

`std::thread` 객체가 join 가능한 상태로 파괴되면 `std::terminate()` 가 호출된다. 직접 수명 관리를 해야 한다면 모든 경로에서 `join()` 또는 `detach()` 가 호출되어야 한다. C++20에서는 자동 합류하는 `std::jthread` 를 우선 고려한다.

10.2 fork와 멀티스레드 프로세스

POSIX 멀티스레드 프로세스가 `fork()` 를 호출하면 자식에는 호출한 스레드만 복제된다. 다른 스레드가 보유하던 mutex 상태는 복사되지만 해당 스레드는 자식에 존재하지 않으므로 교착 상태 위험이 있다.

따라서 멀티스레드 프로그램에서 `fork()` 후 `exec()` 전까지는 POSIX가 허용한 async-signal-safe 함수만 사용하는 것이 원칙이다.

10.3 스레드 지역 저장소

모든 스레드가 공유할 필요가 없는 상태는 `thread_local` 로 분리할 수 있다.

CPP

```
thread_local unsigned request_count = 0;

void handle_request() {
    ++request_count; // 현재 스레드만의 값
}
```

`thread_local` 은 데이터 경쟁을 줄일 수 있지만, 모든 스레드의 값을 합산하거나 일관되게 초기화·정리하는 설계가 추가로 필요할 수 있다.

10.4 스레드 풀과 작업 기반 병렬성

작업마다 새 스레드를 만들면 생성·파괴 비용과 문맥 교환이 증가한다. 스레드 풀은 제한된 수의 작업자 스레드가 큐에서 작업을 가져가도록 한다.

TEXT

```
작업 제출자 → [공유 작업 큐] → 작업자 스레드 1
                                   ↳ 작업자 스레드 2
                                   ↳ 작업자 스레드 N
```

관련 모델에는 스레드 풀, fork-join, OpenMP, 작업 스케줄러, Grand Central Dispatch 등이 있다. 핵심은 스레드 자체보다 수행할 작업을 표현하고 런타임이 실행 자원을 관리하게 하는 것이다.

10.5 취소, 신호와 예외

- 스레드 취소는 자원과 불변식을 안전하게 정리할 수 있는 협력적 방식이 좋다.
- POSIX 신호는 프로세스 전체 또는 특정 스레드에 전달될 수 있어 신호 마스크 설계가 필요하다.
- 한 C++ 스레드에서 처리되지 않은 예외가 스레드 함수 밖으로 빠져나가면 `std::terminate()` 가 호출된다. 필요한 경우 스레드 내부에서 예외를 잡아 `std::exception_ptr` , `std::promise` 또는 `std::future` 로 전달한다.

11. C++ 스레드 생성

C++20에서는 `std::jthread` 를 사용할 수 있다. `std::thread` 와 달리 소멸 시 자동으로 `join()` 하고, 협력적 취소를 위한 중지 토큰도 지원한다.

CPP

```

#include <algorithm>
#include <iostream>
#include <numeric>
#include <thread>
#include <vector>

int main() {
    std::vector<int> values(1'000'000, 1);

    unsigned count =
        std::max(1u, std::thread::hardware_concurrency());

    count = std::min<unsigned>(
        count,
        static_cast<unsigned>(values.size())
    );

    std::vector<long long> partial(count);

    {
        std::vector<std::jthread> workers;
        workers.reserve(count);

        for (unsigned t = 0; t < count; ++t) {
            std::size_t begin = values.size() * t / count;
            std::size_t end = values.size() * (t + 1) / count;

            workers.emplace_back([&, t, begin, end] {
                partial[t] = std::accumulate(
                    values.begin() + begin,
                    values.begin() + end,
                    0LL
                );
            });
        }
    } // 모든 jthread가 join된다.

    long long total =
        std::accumulate(partial.begin(), partial.end(), 0LL);

    std::cout << total << '\n';
}

```

각 스레드는 서로 다른 `partial[t]` 만 변경하므로 논리적인 데이터 경쟁이 없다.

[!warning]

`std::thread::hardware_concurrency()` 는 하드웨어 스레드 수에 대한 힌트일 뿐이다. 0 을 반환할 수 있으며, 반환된 수만큼 스레드를 생성하는 것이 항상 가장 빠르다는 뜻도 아니다.

12. CPU 스케줄링

운영체제의 스케줄러는 준비 상태인 프로세스나 스레드 중 다음 실행 대상을 선택한다.

12.1 스케줄링이 일어나는 시점

스케줄링 결정은 다음과 같은 상태 전이에서 일어난다.

- Running → Waiting: I/O 요청, 조건 대기
- Running → Ready: 타이머 인터럽트에 의한 선점
- Waiting → Ready: I/O 또는 사건 완료
- Running → Terminated: 작업 종료

비선점형 스케줄링은 실행 중인 작업이 스스로 CPU를 반환할 때까지 기다린다. 선점형 스케줄링은 타이머 인터럽트 등을 이용해 운영체제가 실행 중인 작업의 CPU를 회수할 수 있다.

스케줄러가 작업을 선택한 뒤 디스패처(dispatcher)가 문맥 교환, 사용자 모드 전환, 올바른 명령어 위치로의 이동을 수행한다. 실행 중단부터 새 작업이 실제로 실행될 때까지의 시간을 dispatch latency라고 한다.

12.2 주요 평가 기준

- CPU 이용률
- 처리량
- 반환 시간
- 대기 시간
- 응답 시간
- 공정성
- 예측 가능성

주요 시간은 다음처럼 구분한다.

FORMULA · CALCULATION

반환 시간 = 완료 시각 - 도착 시각
 응답 시간 = 최초 실행 시각 - 도착 시각
 대기 시간 = 준비 큐에서 기다린 시간의 합

12.3 대표 알고리즘

정책	핵심	장점	단점
FCFS/FIFO	도착 순서대로 실행	단순함	긴 작업 때문에 짧은 작업이 밀릴 수 있음
SJF	가장 짧은 작업 우선	평균 반환 시간에 유리	실행 시간을 미리 알기 어려움
SRTF	남은 시간이 짧은 작업 선점	짧은 작업에 유리	긴 작업의 기아 가능
Round Robin	일정 시간씩 순환	응답성이 좋음	퀀텀이 작으면 문맥 교환 증가
Priority	높은 우선순위부터 실행	중요 작업 우대	낮은 우선순위의 기아 가능
MLFQ	과거 행동으로 우선순위 변경	대화형 작업에 빠른 응답	정책과 매개변수가 복잡함
비례 배분	CPU 지분을 비율로 배분	자원 비율을 보장	정확한 비율 유지 비용
실시간 스케줄링	마감 시간 중심	시간 제약 처리	분석과 보장이 어려움

우선순위가 낮은 작업의 기아는 오래 기다린 작업의 우선순위를 올리는 aging으로 완화할 수 있다.

MLFQ 핵심 규칙

MLFQ(Multi-Level Feedback Queue)는 작업의 미래 실행 시간을 알 수 없다는 문제를 과거 행동으로 근사한다.

1. 우선순위가 높은 큐의 작업을 먼저 실행한다.
2. 같은 큐에서는 보통 Round Robin을 사용한다.
3. 새 작업은 높은 우선순위에서 시작한다.
4. CPU를 오래 사용한 작업은 낮은 큐로 이동한다.
5. 오래 기다린 작업은 주기적인 priority boost로 기아를 방지한다.
6. 타임 슬라이스가 끝나기 직전에 I/O를 요청하는 방식으로 우선순위를 독점하지 못하도록 누적 CPU 사용량을 계산한다.

12.4 멀티프로세서 스케줄링

멀티코어에서는 단순히 다음 작업을 고르는 것 외에 다음 문제가 추가된다.

- **캐시 친화성**: 이전에 실행한 코어에서 다시 실행하면 따뜻한 캐시를 활용할 수 있다.
- **프로세서 affinity**: 작업을 특정 코어 또는 코어 집합에 묶는다.
- **부하 균등화**: 코어별 실행 큐의 부하를 맞춘다.
- **push migration**: 바쁜 코어의 작업을 다른 코어로 밀어낸다.
- **pull migration/work stealing**: 빈 코어가 다른 큐의 작업을 가져온다.
- **NUMA**: 메모리가 가까운 CPU에서 실행해야 원격 메모리 접근을 줄일 수 있다.
- **SMT**: 하나의 물리 코어가 여러 하드웨어 실행 문맥을 제공하므로 논리 CPU 수와 실제 코어 수가 다를 수 있다.

CPU 중심 작업은 무작정 스레드를 늘리기보다 코어 수와 측정 결과를 기준으로 제한한다. I/O 중심 작업은 대기 시간 때문에 더 많은 동시 작업이 유리할 수 있지만 스레드 과다 생성은 메모리와 문맥 교환 비용을 증가시킨다.

12.5 실시간 스케줄링

- Rate-Monotonic은 주기가 짧은 주기 작업에 더 높은 고정 우선순위를 준다.
- Earliest-Deadline-First는 마감 시간이 가장 가까운 작업을 동적으로 먼저 실행한다.
- soft real-time은 지연을 최소화하지만 마감 준수를 절대 보장하지 않는다.
- hard real-time은 최악 실행 시간과 자원 요구량을 분석하여 마감 준수를 보장해야 한다.

12.6 C++에서 주의할 점

CPP

```
std::thread a(taskA);
std::thread b(taskB);
```

이 코드에서 `taskA()` 가 `taskB()` 보다 먼저 실행된다는 보장은 없다. 실행 순서를 정해야 한다면 스케줄러의 우연한 선택이 아니라 `join()` , 조건 변수, future 등의 동기화 도구를 사용해야 한다.

스레드 우선순위와 CPU affinity는 표준 C++ 기능이 아니다. 프로그램의 정확성을 특정 우선순위나 affinity에 의존시키면 안 된다.

13. 경쟁 조건과 원자성

다음 코드는 안전하지 않다.

CPP

```
int counter = 0;

void work() {
    for (int i = 0; i < 100'000; ++i) {
        ++counter;
    }
}
```

`++counter` 는 개념적으로 다음 연산으로 분리된다.

1. `counter` 를 레지스터로 읽는다.
2. 값을 1 증가시킨다.
3. 결과를 메모리에 저장한다.

두 스레드가 같은 값을 읽은 뒤 각각 증가시켜 저장하면 증가 연산 하나가 사라질 수 있다.

[!important]

C++에서 동기화되지 않은 데이터 경쟁은 단순히 결과가 부정확해지는 문제가 아니라 **undefined behavior**다.

13.1 mutex 사용

CPP

```
#include <mutex>

int counter = 0;
std::mutex counter_mutex;

void work() {
    for (int i = 0; i < 100'000; ++i) {
        std::lock_guard lock(counter_mutex);
        ++counter;
    }
}
```

`std::lock_guard` 는 범위를 벗어날 때 자동으로 mutex를 해제한다. 예외나 조기 `return` 이 발생할 수 있으므로 수동 `lock()` 과 `unlock()` 보다 안전하다.

13.2 atomic 사용

```

CPP
#include <atomic>

std::atomic<int> counter{0};

void work() {
    for (int i = 0; i < 100'000; ++i) {
        counter.fetch_add(1, std::memory_order_relaxed);
    }
}

```

독립적인 통계 카운터처럼 최종 숫자의 원자성만 필요하다면 `memory_order_relaxed` 를 사용할 수 있다. 다른 데이터의 공개와 실행 순서까지 전달해야 한다면 `acquire/release` 또는 기본 순차 일관성을 검토해야 한다.

`std::atomic` 이라고 항상 lock-free인 것은 아니다.

13.3 volatile은 동기화가 아니다

`volatile` 은 멀티스레드 원자성과 실행 순서를 제공하지 않는다. `volatile int counter` 로 바뀌도 위의 경쟁 조건은 해결되지 않는다.

13.4 C++ 메모리 모델과 happens-before

C++에서 데이터 경쟁은 다음 조건을 만족하는 두 메모리 접근이 있을 때 발생한다.

- 서로 다른 스레드에서 실행된다.
- 같은 메모리 위치에 접근한다.
- 하나 이상이 쓰기다.
- 두 접근이 원자 연산이 아니며 happens-before 관계로 정렬되지 않는다.

대표적인 happens-before 관계:

- 스레드를 시작하기 전에 수행한 작업은 새 스레드가 시작한 후 관찰할 수 있다.
- 스레드 안의 작업은 성공한 `join()` 이후의 코드보다 앞선다.
- mutex의 `unlock()` 은 같은 mutex를 이후에 성공적으로 획득한 `lock()` 보다 앞선다.
- release 저장은 그 값을 읽은 acquire 읽기보다 앞선다.

CPP

```

#include <atomic>
#include <cassert>
#include <string>
#include <thread>

std::string payload;
std::atomic<bool> ready{false};

void producer() {
    payload = "complete";
    ready.store(true, std::memory_order_release);
}

void consumer() {
    while (!ready.load(std::memory_order_acquire)) {
        std::this_thread::yield();
    }

    assert(payload == "complete");
}

```

release/acquire는 **ready** 자체의 원자성뿐 아니라 **payload** 기록의 가시성도 전달한다. 단순 카운터에 사용한 **memory_order_relaxed** 에는 이런 데이터 공개 기능이 없다.

[!tip]

메모리 순서를 정확히 증명해야 하는 특별한 이유가 없다면 먼저 mutex나 기본 원자 메모리 순서를 사용한다. 성능 측정 없이 약한 메모리 순서를 선택하면 이해하기 어려운 버그만 늘어날 수 있다.

14. 임계 영역과 락

임계 영역(critical section)은 여러 스레드가 동시에 실행하면 공유 상태가 손상될 수 있는 코드 구간이다.

올바른 임계 영역 해법이 가져야 할 성질:

- **상호 배제**: 한 번에 하나의 스레드만 진입한다.
- **진행**: 임계 영역이 비어 있을 때 진입 결정이 무한히 지연되지 않는다.
- **한정 대기**: 특정 스레드가 무한히 기다리지 않는다.

락 구현의 기반에는 다음과 같은 하드웨어 원자 명령이 있다.

- Test-And-Set
- Compare-And-Swap
- Fetch-And-Add
- Load-Linked/Store-Conditional

14.1 스핀 락

스핀 락은 락이 풀릴 때까지 반복 검사한다. 임계 영역이 매우 짧고 현재 소유자가 곧 실행될 것이 확실한 저수준 환경에서 유용할 수 있다.

일반 응용 프로그램에서는 다음 문제가 발생하기 쉽다.

- 기다리는 동안 CPU를 계속 사용한다.
- 락 소유자가 선점되면 오랫동안 헛돌 수 있다.
- 스레드가 많을수록 메모리 버스와 캐시 경합이 커진다.

따라서 일반적인 C++ 응용 프로그램에서는 먼저 `std::mutex` 를 고려한다.

14.2 락 기반 자료 구조

자료 구조에 락 하나를 추가했다고 모든 사용 방식이 자동으로 안전해지는 것은 아니다.

- **거친 락(coarse-grained lock)**: 자료 구조 전체를 하나의 락으로 보호한다. 정확성은 이해하기 쉽지만 병렬성이 낮다.
- **세밀한 락(fine-grained lock)**: 노드, 버킷 또는 영역별로 락을 둔다. 병렬성은 높지만 락 순서와 객체 수명 관리가 어려워진다.
- **분할 자료 구조**: 해시 테이블의 버킷별 락처럼 서로 독립적인 영역으로 경합을 분산한다.
- **지역 상태 + 주기적 병합**: CPU 또는 스레드별 카운터를 두고 일정 시점에 전역 값으로 합쳐 정확성과 성능을 절충한다.

개별 메서드가 thread-safe여도 여러 메서드의 조합은 원자적이지 않을 수 있다.

CPP

```
if (!queue.empty()) { // 이 검사 직후 다른 스레드가 pop할 수 있음
    auto value = queue.front();
    queue.pop();
}
```

이런 연산은 `try_pop()` 처럼 검사와 제거를 하나의 동기화된 메서드로 제공해야 한다.

인접한 스레드별 카운터는 서로 다른 객체여도 같은 캐시 라인에 놓여 false sharing을 일으킬 수 있다. 데이터 배치와 패딩은 정확성 문제가 아니라 성능 문제이며 반드시 측정 후 조정한다.

15. 조건 변수

조건 변수는 특정 상태가 될 때까지 스레드를 잠들게 한다.

- mutex: 공유 상태에 대한 상호 배제를 제공한다.
- condition variable: 공유 상태가 원하는 조건이 될 때까지 기다린다.

15.1 생산자-소비자 유한 버퍼

CPP · 1/2

```
#include <condition_variable>
#include <deque>
#include <mutex>
#include <optional>
#include <stdexcept>

template<class T>
class BoundedBlockingQueue {
public:
    explicit BoundedBlockingQueue(std::size_t capacity)
        : capacity_(capacity) {
        if (capacity_ == 0) {
            throw std::invalid_argument("capacity must be positive");
        }
    }

    bool push(T value) {
        std::unique_lock lock(mutex_);

        not_full_.wait(lock, [this] {
            return closed_ || queue_.size() < capacity_;
        });

        if (closed_) {
            return false;
        }

        queue_.push_back(std::move(value));
        lock.unlock();

        not_empty_.notify_one();
        return true;
    }

    std::optional<T> pop() {
        std::unique_lock lock(mutex_);

        not_empty_.wait(lock, [this] {
            return closed_ || !queue_.empty();
        });

        if (queue_.empty()) {
            return std::nullopt;
        }
    }
};
```

CPP · 2/2

```

    T value = std::move(queue_.front());
    queue_.pop_front();
    lock.unlock();

    not_full_.notify_one();
    return value;
}

void close() {
    {
        std::lock_guard lock(mutex_);
        closed_ = true;
    }

    not_empty_.notify_all();
    not_full_.notify_all();
}

private:
    std::mutex mutex_;
    std::condition_variable not_empty_;
    std::condition_variable not_full_;
    std::deque<T> queue_;
    const std::size_t capacity_;
    bool closed_ = false;
};

```

이 예제에는 두 조건이 있다.

- 소비자는 큐가 비어 있지 않을 때까지 `not_empty_` 에서 기다린다.
- 생산자는 큐에 빈자리가 생길 때까지 `not_full_` 에서 기다린다.

`close()` 는 양쪽 대기자를 모두 깨워 종료 조건을 다시 검사하게 한다. 알림 전에 mutex를 해제하면 깨어난 스레드가 곧바로 mutex를 획득할 가능성이 높아지지만, 상태 변경 자체는 반드시 mutex를 보유한 상태에서 수행해야 한다.

15.2 조건 변수 규칙

- 조건에 사용되는 상태를 같은 mutex로 보호한다.
- `if` 대신 predicate 또는 `while` 로 조건을 다시 검사한다.
- 깨어났다고 조건이 반드시 참인 것은 아니다.
- `notify_one()` 자체는 프로그램 상태를 저장하지 않는다.
- `wait()` 는 mutex를 원자적으로 해제하고 잠든다.
- 깨어날 때 `wait()` 는 mutex를 다시 획득한 뒤 반환한다.

16. 세마포어

세마포어는 정수 카운터와 대기 큐로 동시 접근 수를 제어한다.

- `acquire()` /wait/P: 값이 양수이면 감소하고 진행하고, 아니면 대기한다.
- `release()` /post/V: 값을 증가시키고 필요한 경우 대기자를 깨운다.

초기값이 1이면 이진 세마포어처럼 사용할 수 있다. 초기값이 N이면 동시에 N개의 스레드만 자원에 접근하도록 제한한다.

C++

```
#include <semaphore>

std::counting_semaphore<3> connection_slots{3};

class Permit {
public:
    explicit Permit(std::counting_semaphore<3>& semaphore)
        : semaphore_(semaphore) {
        semaphore_.acquire();
    }

    ~Permit() {
        semaphore_.release();
    }

    Permit(const Permit&) = delete;
    Permit& operator=(const Permit&) = delete;

private:
    std::counting_semaphore<3>& semaphore_;
};

void use_server() {
    Permit permit(connection_slots);

    perform_request(); // 동시에 최대 3개만 실행
} // 정상 반환과 예외 모두 여기서 release
```

RAII 객체를 사용했기 때문에 `perform_request()` 가 예외를 던져도 permit이 반환된다.

16.1 mutex와 세마포어 차이

- mutex에는 소유자 개념이 있다.
- 일반적으로 mutex를 획득한 스레드가 해제해야 한다.
- 세마포어의 `release()` 는 다른 스레드가 호출할 수 있다.
- mutex는 상호 배제에 적합하다.
- 세마포어는 자원 개수 제한이나 스레드 간 사건 전달에 적합하다.

16.2 고전적인 동기화 문제

유한 버퍼

생산자는 빈 슬롯을 기다리고 소비자는 채워진 슬롯을 기다린다. 버퍼 인덱스와 실제 자료의 변경은 mutex로 보호해야 한다. 빈 슬롯과 채워진 슬롯을 나타내는 세마포어만 사용하고 버퍼 변경을 보호하지 않으면 여러 생산자 또는 소비자가 동시에 같은 위치를 조작할 수 있다.

Readers-Writers

여러 reader는 동시에 읽을 수 있지만 writer는 독점 접근해야 한다.

- reader 선호 정책은 writer 기아를 일으킬 수 있다.
- writer 선호 정책은 reader 지연을 증가시킬 수 있다.
- `std::shared_mutex` 와 `std::shared_lock` 으로 이 패턴을 표현할 수 있지만 읽기 비율, 임계 영역 길이, 구현 비용에 따라 일반 mutex보다 느릴 수도 있다.

식사하는 철학자

모든 참여자가 자원 하나를 먼저 획득하고 다음 자원을 기다리면 환형 대기(死)가 생긴다. 다음 방법으로 풀을 수 있다.

- 자원 획득 순서를 전역적으로 통일한다.
- 마지막 참여자만 반대 순서로 획득한다.
- 동시에 자원을 시도할 수 있는 참여자 수를 제한한다.
- 필요한 자원을 원자적으로 함께 획득한다.

16.3 liveness 문제 구분

문제	상태
교착 상태	관련 스레드가 모두 기다리며 누구도 진행하지 못함
기아	시스템 전체는 진행하지만 특정 스레드가 계속 자원을 얻지 못함
라이브락	스레드들이 상태를 계속 바꾸지만 유용한 작업은 진행되지 않음
우선순위 역전	낮은 우선순위 스레드가 가진 락 때문에 높은 우선순위 스레드가 대기함

우선순위 역전은 priority inheritance처럼 락 소유자의 우선순위를 일시적으로 높이는 방식으로 완화할 수 있다.

17. 교착 상태

교착 상태(deadlock)는 스레드들이 서로가 가진 자원을 기다리며 모두 진행하지 못하는 상태다.

17.1 Coffman의 네 조건

다음 네 조건이 동시에 성립하면 교착 상태가 발생할 수 있다.

1. **상호 배제**: 자원을 한 번에 하나의 스레드만 사용한다.
2. **점유하며 대기**: 자원을 가진 채 다른 자원을 기다린다.
3. **비선점**: 보유한 자원을 강제로 회수할 수 없다.
4. **환형 대기**: 스레드와 자원 사이에 원형 대기 관계가 존재한다.

17.2 교착 상태가 가능한 코드

CPP

```
// 스레드 1
a.lock();
b.lock();

// 스레드 2
b.lock();
a.lock();
```

스레드 1이 **a**, 스레드 2가 **b**를 획득하면 서로 상대방의 락을 기다릴 수 있다.

17.3 std::scoped_lock 사용

CPP

```
#include <mutex>

struct Account {
    int balance;
    std::mutex mutex;
};

void transfer(Account& from, Account& to, int amount) {
    if (&from == &to) {
        return;
    }

    std::scoped_lock lock(from.mutex, to.mutex);

    if (from.balance >= amount) {
        from.balance -= amount;
        to.balance += amount;
    }
}
```

`std::scoped_lock`은 여러 mutex를 교착 상태가 발생하지 않도록 획득한다.

17.4 교착 상태 방지 원칙

- 모든 락에 전역적인 획득 순서를 정한다.
- 락을 가진 채 I/O를 실행하지 않는다.
- 락을 가진 채 외부 콜백을 호출하지 않는다.
- 임계 영역을 짧게 유지한다.
- 중첩 락을 최소화한다.
- 여러 락은 `std::scoped_lock`으로 한꺼번에 획득한다.
- 필요한 경우 `try_lock()`과 재시도를 사용한다.
- 타임아웃은 근본적인 방지책보다 복구 수단으로 본다.

17.5 자원 할당 그래프

자원 할당 그래프는 프로세스와 자원의 관계를 방향 그래프로 표현한다.

TEXT

프로세스 —요청—> 자원
 프로세스 <=할당= 자원

자원 종류마다 인스턴스가 하나뿐이라면 그래프의 사이클은 교착 상태를 의미한다. 인스턴스가 여러 개라면 사이클은 교착 상태의 필요조건일 수 있지만 그 자체로 충분하지는 않다.

17.6 예방과 회피의 차이

- **예방(prevention)**: Coffman 조건 중 하나가 성립하지 않도록 시스템 규칙을 제한한다.
- **회피(avoidance)**: 각 작업의 최대 자원 요구량을 알고, 할당 후에도 안전 상태가 유지될 때만 자원을 준다.

안전 상태는 모든 프로세스가 일정한 순서로 필요한 자원을 얻어 종료할 수 있는 safe sequence가 존재하는 상태다. 불안전 상태가 즉시 교착 상태라는 뜻은 아니지만 미래의 요청에 따라 교착 상태로 진행될 수 있다.

Banker's algorithm은 다음 정보를 이용해 할당 후의 안전 상태를 검사한다.

- 시스템이 보유한 전체 자원 수
- 현재 사용 가능한 자원 수
- 프로세스별 현재 할당량
- 프로세스별 최대 요구량
- $Need = Max - Allocation$

현실의 일반 프로그램은 최대 요구량을 미리 알기 어렵기 때문에 모든 상황에 Banker's algorithm을 사용하지는 않는다.

17.7 탐지

교착 상태를 허용한 뒤 주기적으로 탐지할 수도 있다.

- 자원 종류마다 인스턴스가 하나라면 wait-for graph의 사이클을 탐지한다.
- 인스턴스가 여러 개라면 가용량, 현재 할당량, 남은 요청량을 이용한 탐지 알고리즘이 필요하다.
- 탐지 주기가 짧으면 검사 비용이 증가하고, 길면 교착 상태가 오래 유지되며 더 많은 작업이 얼릴 수 있다.

17.8 복구

교착 상태를 탐지한 뒤 사용할 수 있는 복구 방법:

- 관련 프로세스를 모두 종료한다.
- 하나씩 종료하며 사이클이 사라지는지 검사한다.
- 일부 자원을 선점하고 피해 프로세스를 롤백한다.

희생자를 선택할 때는 우선순위, 지금까지 실행한 시간, 남은 시간, 보유 자원, 종료 비용, 다른 프로세스에 미치는 영향을 함께 고려한다. 같은 프로세스가 계속 희생되지 않도록 롤백 횟수도 비용에 포함해야 한다.

18. 교착 상태 이외의 병행성 오류

18.1 원자성 위반

함께 실행되어야 하는 연산 사이에 다른 스레드가 끼어드는 문제다.

CPP

```
if (pointer != nullptr) {
    use(pointer);
}
```

검사와 사용 사이에 다른 스레드가 객체를 제거할 수 있다. 검사와 사용을 같은 mutex로 보호하거나 `std::shared_ptr` 처럼 객체 수명을 안전하게 관리해야 한다. 단, `shared_ptr` 는 객체의 수명만 보호할 뿐 객체 내부 필드의 동시 읽기/쓰기를 자동으로 thread-safe하게 만들지는 않는다.

18.2 순서 위반

A 작업이 B보다 먼저 실행되어야 하지만 그 순서가 강제되지 않은 문제다.

잘못된 방법:

CPP

```
std::this_thread::sleep_for(std::chrono::milliseconds(100));
// 이제 다른 스레드가 끝났을 것이라고 추측
```

올바른 동기화 도구:

- `join()`
- `std::condition_variable`
- `std::future` 와 `std::promise`
- `std::latch`
- `std::barrier`
- 메시지 큐

`sleep()` 은 실행 순서를 보장하지 않는다.

19. 스레드 종료와 협력적 취소

스레드를 강제로 종료하면 다음 문제가 발생할 수 있다.

- mutex를 가진 채 종료
- 메모리와 파일 누수
- 자료 구조가 절반만 변경된 상태로 남음
- 프로그램의 불변식 훼손

C++20에서는 `std::jthread` 와 `std::stop_token` 을 이용한 협력적 취소를 사용할 수 있다.

CPP

```

#include <chrono>
#include <thread>

using namespace std::chrono_literals;

int main() {
    std::jthread worker([](std::stop_token stop) {
        while (!stop.stop_requested()) {
            perform_one_unit();
        }

        cleanup();
    });

    std::this_thread::sleep_for(1s);
    worker.request_stop();
}

```

스레드가 무한 블로킹 I/O에서 기다리면 중지 요청을 확인하지 못할 수 있다. 이때는 타임아웃, 취소 가능한 I/O, 소켓 종료 같은 운영체제별 방법이 필요하다.

20. 프로세스와 스레드 선택 기준

프로세스가 적합한 경우

- 외부 코드나 플러그인을 격리해야 한다.
- 한 작업의 충돌이 전체 프로그램을 종료시키면 안 된다.
- 서로 다른 보안 권한이 필요하다.
- 독립 서비스로 배포해야 한다.
- 서로 다른 언어나 런타임을 사용한다.

스레드가 적합한 경우

- 큰 자료 구조를 자주 공유한다.
- 통신 지연이 작아야 한다.
- 하나의 작업을 여러 CPU 코어로 병렬화한다.
- 같은 응용 프로그램의 내부 작업이다.

이벤트 기반 구조가 적합한 경우

- 연결 수가 매우 많다.
- 각 요청이 주로 I/O를 기다린다.
- 작업이 짧고 블로킹 호출을 피할 수 있다.

20.1 이벤트 기반 병행성의 구조와 한계

이벤트 기반 프로그램은 보통 이벤트 루프에서 준비된 사건을 가져와 짧은 처리기를 실행한다.

TEXT

```
while (running) {
    events = wait_for_events();
    for (event : events) {
        process(event);
    }
}
```

하나의 이벤트 루프에서 처리기가 순차 실행되면 공유 상태에 대한 락 사용을 줄일 수 있다. 하지만 다음 문제가 남는다.

- 처리기에서 블로킹 시스템 콜을 실행하면 전체 이벤트 루프가 멈춘다.
- CPU 작업이 길면 다른 모든 사건의 응답 시간이 증가한다.
- 비동기 작업 전후의 상태를 명시적으로 저장해야 한다.
- 여러 CPU 코어를 활용하려면 이벤트 루프를 여러 개 두거나 작업을 스레드 풀로 전달해야 한다.
- 파일, 네트워크, 타이머 등 I/O 종류마다 비동기 지원 방식이 다를 수 있다.

따라서 이벤트 처리기는 짧게 유지하고, 블로킹 또는 CPU 중심 작업은 제한된 작업자 풀로 넘기며, 완료 결과는 다시 이벤트 루프로 전달하는 구조가 일반적이다.

실제 서버에서는 다음처럼 여러 모델을 조합할 수 있다.

TEXT

여러 프로세스

- └ 프로세스마다 이벤트 루프
 - └ CPU 작업용 제한된 스레드 풀

21. C++ 멀티스레드 체크리스트

- ☐ 공유 데이터의 소유권이 명확한가?
- ☐ 공유 상태를 메시지 전달로 대체할 수 있는가?
- ☐ 가능하면 불변 데이터를 공유하고 있는가?
- ☐ mutex를 RAII 객체와 함께 사용하는가?
- ☐ `volatile` 을 동기화 수단으로 사용하지 않았는가?
- ☐ `sleep()` 으로 실행 순서를 맞추지 않았는가?
- ☐ 조건 변수를 predicate와 함께 사용하는가?
- ☐ 유한 큐가 가득 찼을 때의 역압력 정책이 정의되어 있는가?
- ☐ 여러 락을 `std::scoped_lock` 으로 획득하는가?
- ☐ 락을 가진 채 I/O나 외부 콜백을 실행하지 않는가?
- ☐ 스레드를 무제한으로 생성하지 않는가?
- ☐ 모든 `std::thread` 가 파괴되기 전에 join 또는 detach되는가?

- ☐ `shared_ptr` 가 객체 수명만 보호하며 내부 데이터 동기화는 별도라는 점을 고려했는가?
- ☐ 멀티스레드 프로세스에서 `fork()` 후 안전하지 않은 함수를 호출하지 않는가?
- ☐ 프로그램의 정확성이 스레드 우선순위에 의존하지 않는가?
- ☐ 오류와 예외 경로에서도 락과 자원이 해제되는가?
- ☐ ThreadSanitizer 같은 도구로 데이터 경쟁을 검사했는가?

22. 핵심 요약

1. 프로세스는 독립된 주소 공간과 자원을 가지며 장애 격리가 좋다.
2. 스레드는 프로세스의 자원을 공유하며 실행 문맥과 스택은 따로 가진다.
3. 운영체제는 타이머 인터럽트와 문맥 교환으로 CPU를 가상화한다.
4. 스케줄링 순서는 비결정적이므로 코드가 특정 실행 순서를 가정하면 안 된다.
5. 공유 데이터를 동기화하지 않고 읽고 쓰면 C++ 데이터 경쟁이 발생한다.
6. 단순 값의 원자적 변경에는 `std::atomic` 을 사용할 수 있다.
7. 복합 불변식 보호에는 mutex가 필요하다.
8. 상태가 바뀔 때까지 기다리려면 조건 변수를 사용한다.
9. 자원의 동시 사용 개수를 제한하려면 세마포어를 사용할 수 있다.
10. 교착 상태는 락 순서를 통일하고 `std::scoped_lock` 을 사용해 예방한다.
11. 회피는 안전 상태를 유지하는 할당만 허용하고, 탐지는 교착 상태를 허용한 뒤 사이클이나 자원 상태를 검사한다.
12. 멀티코어에서는 캐시 친화성, affinity, 부하 균등화, NUMA까지 고려한다.
13. 이벤트 기반 구조도 블로킹 호출과 긴 CPU 작업을 별도 작업자에게 넘겨야 한다.

권장 학습 순서

TEXT

프로세스와 상태

- 제한적 직접 실행과 문맥 교환
- CPU 스케줄링
- 스레드와 공유 메모리
- mutex와 atomic
- 조건 변수와 세마포어
- 교착 상태
- C++ 메모리 모델

관련 인덱스: OS 공부

[!success] 최종 검증 기록

- [ocs.pdf](#) : 프로세스, 스레드, CPU 스케줄링, 동기화 도구·사례, 교착 상태 장을 시각적으로 재대조했다.
- [ostep.pdf](#) : 프로세스 개념·API, 제한적 직접 실행, 스케줄링, 락, 조건 변수, 세마포어, 병행성 오류, 이벤트 기반 병행성 장을 재대조했다.
- C++20의 `std::jthread` , acquire/release, 유한 버퍼, `std::counting_semaphore` , `std::scoped_lock` 예제는 MSVC `/std:c++20 /W4 /permissive-` 설정으로 컴파일 및 실행 검증했다.
- POSIX `fork()` · `pipe()` 예제는 Windows 전용 컴파일러 대상이 아니며 Linux, macOS 또는 WSL에서 컴파일하는 예제다.

title: 동시성과 병렬성 aliases:

tags:

created: 2026-08-01 updated: 2026-08-01

- Concurrency and Parallelism
- 병행성과 병렬성
- CS
- OS
- Concurrency
- Parallelism
- Cpp

동시성과 병렬성

[!summary] 한 문장 정리

동시성은 여러 작업을 함께 진전시키는 구조이고, 병렬성은 여러 작업을 같은 순간에 실제로 실행하는 방식이다.

학습 목표

- 동시성, 병렬성, 비동기의 차이를 설명한다.
- 작업 병렬성, 데이터 병렬성, 파이프라인 병렬성을 구분한다.
- 멀티코어에서 캐시 친화성, 일관성, false sharing과 NUMA가 중요한 이유를 이해한다.
- 데이터 경쟁, 원자성, 순서, 가시성 문제를 구분한다.
- C++ 메모리 모델과 happens-before의 기본을 이해한다.
- mutex, atomic, 조건 변수, 세마포어, barrier, future의 사용 목적을 구분한다.
- 동시성 모델을 성능과 정확성 양쪽에서 평가한다.

용어 안내

concurrency 는 한국어 자료에서 **동시성** 또는 **병행성**으로 번역된다. 이 문서에서는 사용자의 표현에 맞춰 동시성이라고 쓰며, PDF에 등장하는 병행성과 같은 개념으로 취급한다.

- concurrency: 동시성 또는 병행성
- parallelism: 병렬성
- asynchronous: 비동기

참고 자료

- C:\Users\user\Desktop\OS\ocs.pdf
- 스레드와 병행성, CPU 스케줄링, 동기화 도구, 동기화 사례, 교착 상태
- C:\Users\user\Desktop\OS\ostep.pdf
- 멀티프로세서 스케줄링, 병행성 개요, 스레드 API, 락, 병행 자료 구조, 조건 변수, 세마포어, 병행성 오류, 이벤트 기반 병행성

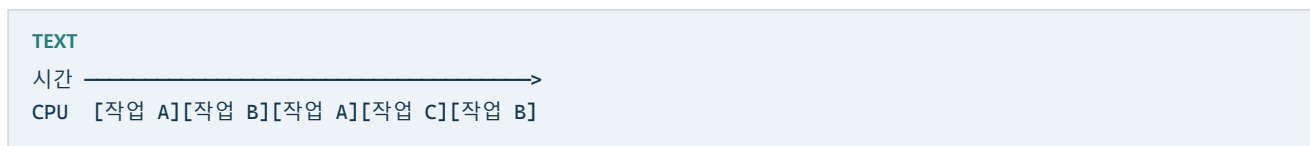
관련 선행 노트: 01.프로세스와 스레드

1. 동시성과 병렬성의 차이

1.1 동시성

동시성(concurrency)은 둘 이상의 작업이 같은 시간 구간 안에서 진행되는 성질이다. 작업들이 반드시 같은 순간에 실행될 필요는 없다.

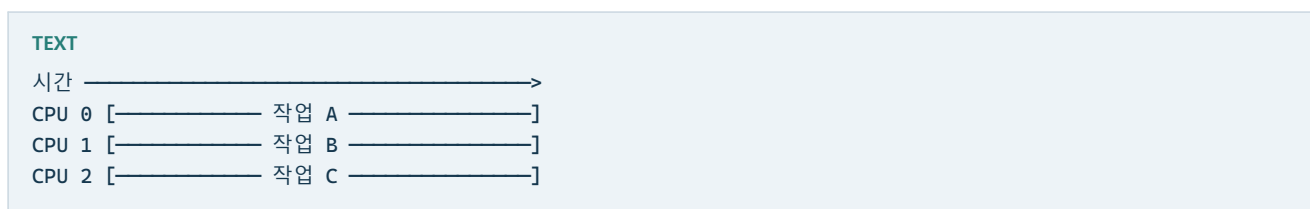
단일 코어에서도 운영체제가 실행 대상을 빠르게 바꾸면 여러 작업이 함께 진행되는 것처럼 보인다.



이 경우 한 순간에는 하나의 작업만 실행되지만 A, B, C의 실행 기간은 서로 겹친다.

1.2 병렬성

병렬성(parallelism)은 둘 이상의 작업이 여러 실행 자원에서 같은 순간에 실제로 실행되는 성질이다.



병렬 실행에는 여러 CPU 코어, 여러 프로세서, GPU 실행 유닛 또는 분산된 컴퓨터가 필요하다.

1.3 관계

상황	동시성	병렬성
단일 코어에서 여러 스레드를 번갈아 실행	O	X
이벤트 루프가 여러 네트워크 요청을 처리	O	반드시 그렇지 않음
여러 코어가 배열의 서로 다른 구간을 계산	O	O
한 스레드가 순차적으로 하나의 작업만 수행	X	X

넓은 의미에서는 병렬 실행도 여러 작업이 함께 진행되므로 동시성에 포함된다. 하지만 설계 관점에서는 다음처럼 목적이 다르다.

- 동시성의 중심 질문: 여러 작업의 상호작용과 대기를 어떻게 올바르게 다룰 것인가?
- 병렬성의 중심 질문: 하나의 일을 어떻게 나누어 동시에 더 빨리 끝낼 것인가?

1.4 비결정성과 interleaving

동시 작업의 실제 실행 순서는 스케줄러, 인터럽트, 코어 수와 시스템 부하에 따라 달라진다. 같은 입력으로 실행해도 명령어가 섞이는 순서(interleaving)가 달라질 수 있다.

올바른 동시성 프로그램은 특정 스레드가 먼저 실행될 것이라고 가정하지 않고, 동기화 규칙이 허용하는 모든 실행 순서에서 불변식을 지켜야 한다.

[!important]

동시성은 프로그램 구조에 관한 성질이고, 병렬성은 실행 시점에 실제 자원을 사용하는 방식에 가깝다.

2. 비동기와 동시성

비동기(asynchronous)는 호출자가 작업 완료를 기다리지 않고 제어권을 돌려받는 인터페이스 또는 실행 방식이다.

TEXT

요청 시작 → 즉시 반환 → 다른 작업 수행 → 완료 통지

비동기는 동시성과 관련 있지만 같은 개념은 아니다.

- 단일 스레드 이벤트 루프도 비동기 I/O로 동시성을 제공할 수 있다.
- 비동기 API가 내부적으로 별도 스레드를 사용할 수도 있다.
- 비동기라고 해서 반드시 여러 코어에서 병렬 실행되는 것은 아니다.
- 여러 스레드를 사용해도 호출자가 즉시 결과를 기다리면 비동기성이 거의 드러나지 않을 수 있다.

CPP

```
auto future = std::async(std::launch::async, calculate);

do_other_work();

auto result = future.get();
```

`get()` 을 바로 호출하면 계산과 다른 작업을 겹칠 기회를 잃는다.

3. 동시성이 필요한 이유

3.1 응답성

오래 걸리는 작업과 사용자 입력 처리를 분리하면 작업이 진행되는 동안 UI가 멈추지 않는다.

3.2 지연 시간 숨기기

한 작업이 디스크, 네트워크 또는 외부 장치를 기다리는 동안 다른 작업을 실행한다.

TEXT

작업 A: [CPU][—— I/O 대기 ——][CPU]
 작업 B: [CPU][CPU][CPU]

3.3 처리량 향상

서로 독립적인 요청을 겹쳐 처리하면 단위 시간당 더 많은 요청을 완료할 수 있다.

3.4 멀티코어 활용

CPU 중심 작업을 독립적인 부분으로 나누어 여러 코어에 배치하면 완료 시간을 줄일 수 있다.

3.5 자연스러운 문제 구조 표현

웹 서버의 연결, 게임의 여러 시스템, GUI 이벤트, 생산자와 소비자처럼 본질적으로 독립적인 활동을 별도 작업으로 표현할 수 있다.

[!warning]

동시성 자체가 자동으로 성능을 높이지는 않는다. 작업 생성, 문맥 교환, 동기화, 캐시 미스와 통신 비용이 실제 이득보다 클 수 있다.

4. 작업을 나누는 방법

4.1 작업 병렬성

서로 다른 종류의 작업을 동시에 실행한다.

TEXT

작업 1: 네트워크 요청 수신
 작업 2: 데이터 파싱
 작업 3: 로그 기록

각 작업의 처리 시간과 자원 요구가 다르므로 파이프라인이나 작업 큐와 함께 사용하는 경우가 많다.

4.2 데이터 병렬성

같은 연산을 서로 다른 데이터 구간에 적용한다.

TEXT

배열: [0 ... 249][250 ... 499][500 ... 749][750 ... 999]
 코어: 0 1 2 3

이미지 처리, 벡터 연산, 통계 계산처럼 요소 간 의존성이 적은 작업에 적합하다.

4.3 파이프라인 병렬성

작업을 여러 단계로 나누고 각 단계가 서로 다른 입력을 동시에 처리한다.

TEXT

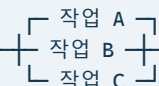
입력 → [읽기] → [파싱] → [계산] → [저장]

파이프라인 처리량은 가장 느린 단계에 의해 제한된다. 단계 사이에는 유한 큐를 두어 무제한 메모리 증가를 방지하고 역압력(backpressure)을 전달해야 한다.

4.4 fork-join

큰 작업을 여러 하위 작업으로 나누어 실행한 뒤 결과를 합친다.

TEXT

큰 작업 - fork  join → 결과

분할 정복, 병렬 합계, 병렬 정렬 등에 적합하다.

4.5 데이터 분해와 기능 분해

- 데이터 분해: 데이터 영역을 여러 작업에 나눈다.
- 기능 분해: 서로 다른 기능을 별도 작업으로 나눈다.

데이터 분해는 부하 균등화가 비교적 쉽지만 경계 데이터의 의존성이 문제가 될 수 있다. 기능 분해는 구조가 자연스럽게 단계를 처리량 차이 때문에 병목이 생길 수 있다.

4.6 멀티코어 프로그래밍의 다섯 과제

1. **작업 식별**: 서로 독립적으로 실행할 수 있는 부분을 찾아야 한다.
2. **균형**: 각 작업의 실행량이 비슷해야 일부 코어만 오래 일하는 상황을 줄일 수 있다.
3. **데이터 분할**: 작업이 처리할 데이터를 충돌 없이 나누어야 한다.
4. **데이터 의존성**: 한 작업의 입력이 다른 작업의 출력에 의존하는지 확인해야 한다.
5. **테스트와 디버깅**: 가능한 실행 순서가 많고 재현이 어려우므로 순차 프로그램과 다른 검증 전략이 필요하다.

5. 동시성 실행 모델

5.1 프로세스 기반

각 작업을 독립된 프로세스로 실행한다.

- 주소 공간이 분리되어 장애 격리가 좋다.
- 통신에는 파이프, 소켓, 공유 메모리 같은 IPC가 필요하다.
- 프로세스 생성과 전환 비용이 상대적으로 크다.

5.2 스레드 기반

하나의 프로세스 안에서 여러 스레드가 주소 공간을 공유한다.

- 데이터 공유와 통신이 빠르다.
- 공유 상태 때문에 데이터 경쟁과 교착 상태 위험이 크다.
- 한 스레드의 메모리 오류가 프로세스 전체에 영향을 줄 수 있다.

5.3 작업 기반

프로그래머는 스레드보다 수행할 작업을 표현하고 런타임이 작업을 제한된 작업자에게 배치한다.

- 스레드 풀
- fork-join 프레임워크
- work stealing 스케줄러
- OpenMP와 작업 그래프

작업 기반 모델은 스레드의 생성·소멸을 반복하지 않고 실행 자원의 수를 통제하기 쉽다.

5.4 이벤트 기반

이벤트 루프가 준비된 사건을 하나씩 가져와 처리한다.

TEXT

```
while (running) {
    events = wait_for_events();
    for (event : events) {
        process(event);
    }
}
```

단일 이벤트 루프는 매 순간 하나의 처리기만 실행하므로 공유 상태의 락을 줄일 수 있다. 그러나 처리기에서 블로킹 호출이나 긴 CPU 계산을 실행하면 전체 이벤트 루프가 멈춘다.

5.5 코루틴

코루틴은 실행을 중단하고 나중에 재개할 수 있는 함수다. 비동기 흐름을 순차 코드처럼 표현하는 데 도움이 되지만 코루틴 자체가 스레드는 아니며 병렬 실행을 자동으로 제공하지 않는다.

[!important]

실행 모델과 실행 자원을 구분해야 한다. 코루틴 수천 개가 하나의 스레드에서 동작할 수도 있고, 작업 런타임이 여러 스레드에 작업을 병렬 배치할 수도 있다.

6. 멀티코어 하드웨어에서 생기는 문제

6.1 코어와 하드웨어 스레드

물리 코어는 실제 실행 자원을 제공한다. SMT 또는 Hyper-Threading은 한 물리 코어가 여러 논리 CPU 문맥을 노출하게 한다.

논리 CPU가 두 배라고 CPU 계산 성능이 정확히 두 배가 되는 것은 아니다. 논리 CPU들은 실행 유닛과 캐시 같은 자원을 일부 공유한다.

6.2 캐시 계층

CPU는 메인 메모리보다 훨씬 빠른 캐시를 사용한다.

TEXT

레지스터 → L1 → L2 → L3 → 메인 메모리

빠름	느림
작음	큼

병렬 프로그램의 성능은 연산 수뿐 아니라 데이터가 어느 캐시에 존재하는지에 크게 영향을 받는다.

6.3 캐시 일관성

여러 코어가 같은 메모리를 캐시에 보관할 수 있으므로 하드웨어는 캐시 일관성 프로토콜로 동일한 메모리 위치의 변경을 전달한다.

캐시 일관성은 같은 메모리 위치의 값이 모순되지 않도록 돕지만 다음을 자동으로 해결하지는 않는다.

- 프로그램의 데이터 경쟁
- 여러 연산의 원자성
- 원하는 실행 순서
- 고수준 자료 구조의 불변식

6.4 캐시 일관성과 메모리 일관성

- cache coherence는 주로 **하나의 메모리 위치**에 대한 여러 코어의 읽기·쓰기 값이 어떻게 일치하는지를 다룬다.
- memory consistency는 **서로 다른 메모리 위치를 포함한 여러 연산의 관찰 순서**를 다룬다.

하드웨어의 캐시 일관성 프로토콜이 있어도 다른 스레드가 모든 쓰기를 소스 코드 순서대로 관찰한다고 보장되지는 않는다. C++ 메모리 모델의 mutex, atomic, acquire/release 같은 동기화가 필요한 이유다.

6.5 false sharing

서로 다른 스레드가 논리적으로 다른 변수만 변경하더라도 변수가 같은 캐시 라인에 있으면 캐시 라인이 코어 사이에서 계속 이동할 수 있다.

CPP

```
#include <atomic>

struct alignas(64) PaddedCounter {
    std::atomic<long long> value{0};
};

PaddedCounter counters[8];
```

64 는 흔한 캐시 라인 크기를 가정한 교육용 예시다. 실제 값과 효과는 하드웨어 및 데이터 배치에 따라 다르므로 반드시 측정해야 한다.

6.6 NUMA

NUMA 시스템에서는 CPU와 가까운 메모리와 멀리 있는 메모리의 접근 비용이 다르다. 작업을 다른 NUMA 노드로 이동시키면 데이터 접근 비용이 증가할 수 있다.

따라서 대규모 병렬 프로그램은 다음을 함께 고려해야 한다.

- 작업을 어느 CPU에 배치하는가?
- 데이터는 어느 NUMA 노드에 할당되어 있는가?
- 작업을 이동할 것인가, 데이터를 이동할 것인가?

7. 작업 배치와 스케줄링

7.1 단일 작업 큐

OSTEP은 이 구조를 SQMS(Single-Queue Multiprocessor Scheduling)라고 설명한다.

모든 작업자가 하나의 큐에서 작업을 가져온다.

장점:

- 구현이 단순하다.
- 전체 부하를 균등하게 나누기 쉽다.

단점:

- 큐의 락과 캐시 라인에 경합이 집중된다.
- 작업이 여러 CPU를 이동해 캐시 친화성이 낮아질 수 있다.

7.2 작업자별 큐

OSTEP은 CPU별로 큐를 두는 구조를 MQMS(Multi-Queue Multiprocessor Scheduling)라고 설명한다.

각 작업자가 자신의 큐를 사용한다.

장점:

- 공유 큐 경합이 줄어든다.
- 작업이 같은 CPU에서 실행되어 캐시를 재사용하기 쉽다.

단점:

- 작업 분배가 잘못되면 어떤 CPU는 놀고 다른 CPU는 바빠진다.

7.3 work stealing

할 일이 없는 작업자가 다른 작업자의 큐에서 작업을 가져온다.

TEXT

Worker 0: [A][B][C][D]

Worker 1: [] — steal —> [D]

도둑질 빈도가 너무 높으면 큐 검사와 동기화 비용이 증가하고, 너무 낮으면 부하 불균형이 오래 유지된다.

7.4 affinity와 migration

- affinity는 작업을 이전에 실행한 CPU에 유지하여 캐시 재사용을 높인다.
- migration은 작업을 다른 CPU로 옮겨 부하 불균형을 해소한다.
- 두 목표는 충돌할 수 있으므로 스케줄러가 캐시 친화성과 균형 사이에서 절충한다.

7.5 oversubscription

실행 가능한 CPU 중심 스레드가 논리 CPU보다 훨씬 많으면 문맥 교환과 캐시 손실이 늘어난다.

- CPU 중심 작업: 보통 사용 가능한 코어 수 근처에서 시작해 측정한다.
- I/O 중심 작업: 대기 시간이 길어 더 많은 동시 작업이 유리할 수 있다.
- 스레드 수는 하드웨어 수만이 아니라 작업의 CPU/I/O 비율과 외부 제한을 반영해야 한다.

8. 동시성 정확성의 세 문제

8.1 원자성

여러 연산이 다른 스레드가 끼어들 수 없는 하나의 단위처럼 실행되어야 하는 문제다.

CPP

```
if (balance >= amount) {
    balance -= amount;
}
```

검사와 변경 사이에 다른 스레드가 끼어들면 잔액 불변식이 깨질 수 있다.

8.2 순서

A가 B보다 먼저 일어나야 하지만 그 순서가 강제되지 않은 문제다.

TEXT

생산자: 데이터 작성 → 준비 완료 표시
소비자: 준비 완료 확인 → 데이터 사용

준비 표시와 데이터 가시성 사이에 올바른 동기화가 필요하다.

8.3 가시성

한 코어의 스레드가 변경한 값을 다른 코어의 스레드가 언제 관찰할 수 있는가에 관한 문제다. 컴파일러 최적화, CPU의 메모리 순서, 캐시 때문에 소스 코드의 단순한 줄 순서만으로는 스레드 간 가시성이 보장되지 않는다.

8.4 데이터 경쟁

C++에서 다음 조건을 만족하면 데이터 경쟁(data race)이 발생한다.

- 서로 다른 스레드가 같은 메모리 위치에 접근한다.
- 접근 중 하나 이상이 쓰기다.
- 원자 연산이 아니며 happens-before로 정렬되지 않는다.

C++의 데이터 경쟁은 undefined behavior다.

[!note] 실제 오류가 주는 교훈

OSTEP이 소개한 실제 소프트웨어 연구에서는 조사된 비교착 상태 병행성 오류의 대부분이 원자성 위반 또는 순서 위반에 속했다. 교착 상태만 찾는 것으로는 동시성 오류를 충분히 예방할 수 없다.

9. C++ 메모리 모델

9.1 happens-before

happens-before는 한 스레드의 연산 결과가 다른 연산에 관찰 가능하도록 순서를 연결하는 관계다. 단순한 실제 시각의 선후 관계보다 강한 의미를 가진다.

대표적인 관계:

- 스레드 생성 전의 연산은 새 스레드 시작 이후보다 앞선다.
- 스레드 내부 연산은 성공한 `join()` 이후보다 앞선다.
- mutex의 `unlock()` 은 같은 mutex를 이후에 획득한 `lock()` 보다 앞선다.
- release 저장은 그 값을 관찰한 acquire 읽기보다 앞선다.
- barrier의 한 단계 도착 전 연산은 다음 단계 진행 이후에 관찰 가능하다.

9.2 release와 acquire

CPP

```
#include <atomic>
#include <cassert>
#include <string>
#include <thread>

std::string payload;
std::atomic<bool> ready{false};

void producer() {
    payload = "complete";
    ready.store(true, std::memory_order_release);
}

void consumer() {
    while (!ready.load(std::memory_order_acquire)) {
        std::this_thread::yield();
    }

    assert(payload == "complete");
}
```

consumer가 release 저장을 acquire로 관찰하면 그보다 앞서 수행된 `payload` 기록도 관찰할 수 있다.

9.3 relaxed

`memory_order_relaxed` 는 해당 원자 객체의 원자성만 보장한다. 다른 메모리 위치의 변경을 함께 공개하는 순서는 제공하지 않는다.

독립적인 통계 카운터처럼 다른 데이터와 순서 관계가 필요하지 않은 경우에 적합할 수 있다.

9.4 volatile

`volatile` 은 멀티스레드 동기화 도구가 아니다. 메모리 맵 I/O나 구현이 정의한 특별한 접근에는 쓰일 수 있지만 C++ 스레드 사이의 원자성이나 happens-before를 만들지 않는다.

[!tip]

악한 메모리 순서를 직접 사용하기 전에 mutex나 기본 순차 일관성 원자로 정확하게 구현하고 성능을 측정한다.

10. 동기화 도구 선택

도구	해결하려는 문제	대표 용도
<code>std::mutex</code>	복합 공유 상태의 상호 배제	컨테이너와 여러 필드의 불변식 보호
<code>std::atomic</code>	단일 값의 원자적 접근	카운터, 상태 플래그, 저수준 동기화
<code>std::condition_variable</code>	조건이 참이 될 때까지 대기	생산자-소비자, 작업 큐
<code>std::counting_semaphore</code>	사용 가능한 자원 수 관리	연결 수, 버퍼 슬롯 제한
<code>std::latch</code>	정해진 횟수의 완료를 한 번 기다림	시작 또는 완료 게이트
<code>std::barrier</code>	여러 스레드가 반복적으로 단계 동기화	반복 계산의 phase 경계
<code>std::future</code> / <code>promise</code>	결과 또는 예외를 전달	비동기 작업 결과 회수

10.1 mutex

복합 연산과 여러 필드로 이루어진 불변식을 보호할 때 사용한다.

CPP

```
std::mutex mutex;

{
    std::lock_guard lock(mutex);
    update_shared_state();
}
```

10.2 atomic

단일 값의 독립적인 원자 변경에는 `atomic`이 더 단순할 수 있다.

CPP

```
std::atomic<unsigned long long> completed{0};

void finish_one() {
    completed.fetch_add(1, std::memory_order_relaxed);
}
```

`atomic` 하나로 여러 필드 사이의 불변식을 자동으로 보호할 수는 없다.

10.3 조건 변수

조건 변수는 다른 스레드가 상태를 변경할 때까지 CPU를 점유하지 않고 기다리게 한다.

CPP

```
std::unique_lock lock(mutex);
condition.wait(lock, [&] { return ready || stopped; });
```

조건은 반드시 predicate 또는 `while` 로 다시 검사한다.

10.4 semaphore

동시에 접근할 수 있는 작업 수를 제한하거나 사건의 개수를 전달한다.

CPP

```
std::counting_semaphore<8> slots{8};
```

10.5 latch와 barrier

- latch는 카운트가 0이 된 뒤 다시 사용할 수 없다.
- barrier는 여러 단계에 반복해서 사용할 수 있다.

10.6 blocking과 spinning

- blocking lock은 락을 얻을 수 없을 때 대기 스레드를 재우므로 CPU를 다른 작업에 사용할 수 있다.
- spin lock은 락을 반복 검사하므로 문맥 교환은 피할 수 있지만 기다리는 동안 CPU를 소비한다.

스핀은 임계 영역이 매우 짧고 락 소유자가 실제로 다른 코어에서 곧 실행을 마칠 때만 유리할 수 있다. 일반 C++ 응용 프로그램에서는 먼저 `std::mutex` 를 사용하고 측정 결과가 있을 때만 다른 선택을 검토한다.

락의 정확성 외에도 공정성을 고려해야 한다. 특정 스레드가 계속 락을 얻지 못하면 기아가 발생할 수 있다. 반대로 강한 공정성 보장은 처리량이나 캐시 지역성을 낮출 수 있다.

11. C++ 병렬 합계 예제

다음 예제는 배열을 둘로 나누어 한쪽은 비동기 작업으로, 다른 쪽은 현재 스레드에서 계산한다.

CPP

```

#include <future>
#include <numeric>
#include <span>

long long parallel_sum(std::span<const int> values) {
    constexpr std::size_t threshold = 50'000;

    if (values.size() <= threshold) {
        return std::accumulate(values.begin(), values.end(), 0LL);
    }

    const std::size_t middle = values.size() / 2;
    auto left_values = values.first(middle);
    auto right_values = values.subspan(middle);

    auto left = std::async(
        std::launch::async,
        [left_values] {
            return parallel_sum(left_values);
        }
    );

    long long right_result = parallel_sum(right_values);
    return left.get() + right_result;
}

```

핵심 사항:

- `std::launch::async` 를 명시해 별도 비동기 실행을 요청한다.
- 작은 구간은 순차 실행해 작업 생성 비용을 제한한다.
- 서로 겹치지 않는 읽기 전용 구간을 사용하므로 데이터 경쟁이 없다.
- threshold가 너무 작으면 작업이 과도하게 생성될 수 있다.
- 실제 서비스에서는 스레드 수를 제어하는 작업 풀이나 검증된 병렬 런타임이 더 적합하다.

사용 예:

CPP

```

#include <iostream>
#include <vector>

int main() {
    std::vector<int> values(1'000'000, 1);
    std::cout << parallel_sum(values) << '\n';
}

```

12. C++ 단계 병렬화 예제

`std::barrier` 는 여러 스레드가 현재 단계를 모두 마친 뒤 다음 단계로 진행하게 한다.

CPP

```

#include <barrier>
#include <cassert>
#include <thread>
#include <vector>

int main() {
    constexpr int worker_count = 4;
    std::vector<int> values(worker_count, 0);

    std::barrier phase(worker_count);
    std::vector<std::jthread> workers;

    for (int id = 0; id < worker_count; ++id) {
        workers.emplace_back([&, id] {
            values[id] = id + 1;        // 1단계
            phase.arrive_and_wait();

            values[id] *= 10;           // 2단계
            phase.arrive_and_wait();
        });
    }

    workers.clear(); // jthread를 join

    assert((values == std::vector<int>{10, 20, 30, 40}));
}

```

barrier는 단계 경계를 만들지만 각 단계 안에서 여러 스레드가 같은 요소를 변경하면 별도의 동기화가 필요하다.

13. 생산자-소비자와 역압력

생산자와 소비자의 속도가 다르면 중간 큐가 필요하다.

TEXT

생산자 → [유한 작업 큐] → 소비자

큐가 무한히 커질 수 있게 두면 소비자가 느릴 때 메모리가 계속 증가한다. 유한 큐는 가득 찼을 때 생산자를 대기시키거나 다음 정책 중 하나를 적용한다.

- 새 작업 거부
- 오래된 작업 제거
- 낮은 우선순위 작업 제거
- 호출자에게 부하를 되돌리는 역압력
- 디스크나 외부 저장소로 넘김

정책은 데이터 손실 가능성, 지연 시간, 처리량, 메모리 제한에 맞춰 선택해야 한다.

구현 시 필요한 두 조건:

- 소비자는 큐가 비어 있지 않을 때까지 기다린다.
- 생산자는 큐에 빈 슬롯이 생길 때까지 기다린다.

구체적인 C++ 유한 버퍼 구현은 01.프로세스와 스레드#15.1 생산자-소비자 유한 버퍼를 참고한다.

13.1 Readers-Writers

여러 reader는 동시에 공유 데이터를 읽을 수 있지만 writer는 독점 접근해야 한다. reader 선호 정책은 writer 기아를, writer 선호 정책은 reader 지연을 일으킬 수 있다. 읽기가 많다고 항상 `std::shared_mutex` 가 더 빠른 것은 아니므로 임계 영역 길이와 경합을 측정해야 한다.

13.2 식사하는 철학자

각 작업이 자원 하나를 보유한 채 다음 자원을 기다리면 환경 대기가 만들어진다. 자원 획득 순서를 통일하거나 필요한 자원을 함께 획득하여 교착 상태를 예방할 수 있다.

14. 이벤트 기반 동시성

14.1 장점

- 단일 이벤트 루프에서는 처리기 사이의 공유 상태 동기화를 줄일 수 있다.
- 많은 연결마다 스레드를 하나씩 만들 필요가 없다.
- 프로그램이 어떤 사건을 언제 처리할지 비교적 직접 제어할 수 있다.

14.2 블로킹 호출 문제

하나의 처리기가 파일 I/O나 긴 계산으로 멈추면 전체 이벤트 루프가 진행하지 못한다.

해결 방법:

- non-blocking I/O
- asynchronous I/O
- readiness notification
- 블로킹 작업 전용 스레드 풀
- CPU 작업 전용 작업자 풀

명시적으로 호출한 I/O만 블로킹 원인이 되는 것은 아니다. 이벤트 처리 중 페이지 폴트가 발생하거나, 이전에는 non-blocking이던 라이브러리 함수가 구현 변경으로 블로킹 동작을 하게 되어도 이벤트 루프 전체가 지연될 수 있다.

14.3 상태 관리

스레드 기반 코드에서는 호출 스택과 지역 변수가 중단된 작업의 상태를 자연스럽게 보관한다. 이벤트 기반 코드에서는 완료 후 이어서 실행할 상태를 별도 객체나 continuation으로 보존해야 한다.

코루틴은 이러한 상태 기계를 컴파일러가 생성하도록 도와주지만, 공유 상태와 병렬 실행의 동기화 문제를 없애지는 않는다.

14.4 멀티코어 이벤트 루프

여러 코어를 사용하려고 이벤트 루프를 여러 개 실행하면 처리기들이 동시에 공유 상태에 접근할 수 있으므로 다시 동기화가 필요하다.

실제 서버는 다음과 같은 하이브리드 구조를 자주 사용한다.

TEXT

프로세스 여러 개

└ 프로세스별 이벤트 루프

└ 제한된 CPU/블로킹 작업자 풀

15. 확장성과 Amdahl의 법칙

프로그램의 일부만 병렬화할 수 있다면 코어 수를 늘려도 최대 성능 향상에는 한계가 있다.

병렬화 가능한 비율을 P , 실행 자원 수를 N 이라고 하면 이상적인 최대 속도 향상은 다음과 같다.

$$S(N) = \frac{1}{(1 - P) + \frac{P}{N}}$$

프로그램의 90%를 병렬화하고 8개 실행 자원을 사용한다면:

$$S(8) = \frac{1}{0.1 + \frac{0.9}{8}} \approx 4.71$$

코어는 8개지만 최대 속도 향상은 약 4.71배다. 실제로는 작업 생성, 통신, 동기화와 캐시 비용 때문에 더 낮다.

15.1 작업량과 임계 경로

- work: 모든 연산을 순차 실행했을 때의 총 작업량
- span: 의존성 때문에 반드시 순서대로 실행해야 하는 가장 긴 경로

이론적인 병렬성은 대략 $\text{work} / \text{span}$ 으로 볼 수 있다. 코어를 늘려도 임계 경로보다 빠르게 완료할 수는 없다.

15.2 강한 확장과 약한 확장

- strong scaling: 문제 크기를 고정하고 실행 자원을 늘렸을 때 얼마나 빨라지는가?
- weak scaling: 실행 자원과 함께 문제 크기도 늘렸을 때 실행 시간을 유지하는가?

16. 병렬 성능을 방해하는 요소

16.1 너무 작은 작업

작업 시간이 생성과 스케줄링 비용보다 짧으면 병렬화가 더 느릴 수 있다. 관련 연산을 묶어 적절한 grain size를 만든다.

16.2 락 경합

모든 작업자가 하나의 전역 락을 자주 획득하면 실질적으로 직렬 실행된다.

개선 방향:

- 임계 영역 축소
- 읽기 전용 데이터 사용
- 데이터 분할
- 지역 결과를 계산한 후 마지막에 병합
- 버킷별 또는 샤드별 락

16.3 부하 불균형

가장 늦게 끝나는 작업이 전체 완료 시간을 결정한다.

- 작업을 더 작은 단위로 나눈다.
- 동적 스케줄링이나 work stealing을 사용한다.
- 입력 데이터에 따른 비용 차이를 측정한다.

16.4 메모리 대역폭

모든 코어가 메모리에서 많은 데이터를 읽으면 연산 장치보다 메모리 대역폭이 먼저 포화될 수 있다. 이런 작업은 코어를 늘려도 성능이 거의 증가하지 않는다.

16.5 캐시와 false sharing

데이터 지역성이 나쁘거나 쓰기 캐시 라인이 계속 이동하면 코어 수 증가가 오히려 성능을 낮출 수 있다.

16.6 순차 병목

직렬 초기화, 최종 병합, 전역 로그, 단일 큐, 단일 파일 출력이 병렬 부분의 이득을 제한할 수 있다.

17. liveness 문제

문제	의미
교착 상태	여러 작업이 서로를 기다리며 모두 멈춤
기아	시스템은 진행하지만 특정 작업이 계속 기회를 얻지 못함
라이브락	작업들이 계속 반응하지만 유용한 진전이 없음
우선순위 역전	낮은 우선순위 작업이 가진 자원 때문에 높은 우선순위 작업이 대기

17.1 교착 상태 조건

다음 네 조건이 동시에 성립하면 교착 상태가 발생할 수 있다.

1. 상호 배제
2. 점유하며 대기
3. 비선점
4. 환형 대기

여러 mutex가 필요하다면 `std::scoped_lock` 으로 함께 획득하거나 시스템 전체에서 일관된 락 순서를 사용한다.

CPP

```
std::scoped_lock lock(first_mutex, second_mutex);
```

17.2 라이브락 예시

두 작업이 충돌을 피하려고 동시에 양보하고 다시 동시에 재시도하는 패턴이다. 무작위 backoff, 우선순위 또는 중앙 조정으로 대칭을 깨뜨릴 수 있다.

18. 동시성 모델 선택 기준

상황	우선 고려할 모델
독립 작업의 장애 격리가 중요	다중 프로세스
큰 공유 자료 구조를 낮은 지연으로 접근	제한된 스레드 또는 작업 풀
CPU 중심 데이터 처리	데이터 병렬성, fork-join
많은 네트워크 연결과 짧은 처리	이벤트 루프, 비동기 I/O
블로킹 라이브러리를 많이 사용	제한된 스레드 풀
여러 단계의 연속 처리	유한 큐를 둔 파이프라인
결과 하나를 나중에 회수	future/promise 또는 task
반복 계산의 단계 경계	barrier

선택 질문:

1. 작업은 CPU 중심인가, I/O 중심인가?
2. 작업끼리 어떤 데이터를 공유하는가?
3. 순서 의존성과 임계 경로는 무엇인가?
4. 장애를 어느 단위까지 격리해야 하는가?
5. 최대 동시 작업 수와 메모리 한도는 얼마인가?
6. 지연 시간과 처리량 중 어느 쪽이 더 중요한가?

19. 측정 방법

19.1 기준선

먼저 정확한 순차 버전의 실행 시간을 측정한다. 병렬 버전은 같은 결과를 생성하는지 확인한 뒤 비교한다.

19.2 측정 항목

- 전체 실행 시간
- 처리량과 지연 시간 분포
- CPU 이용률
- 실행 가능한 스레드 수
- 문맥 교환 수
- 락 대기 시간
- 캐시 미스
- 메모리 대역폭
- 작업 큐 길이
- 코어별 부하

19.3 speedup과 efficiency

FORMULA · CALCULATION

$\text{speedup}(N) = \text{순차 실행 시간} / N\text{개 자원 실행 시간}$

$\text{efficiency}(N) = \text{speedup}(N) / N$

효율이 코어 수 증가와 함께 빠르게 낮아지면 순차 병목, 동기화, 부하 불균형 또는 메모리 병목을 찾아야 한다.

19.4 벤치마크 주의사항

- release 최적화 빌드도 측정한다.
- 충분히 워밍업한다.
- 여러 번 반복하고 분포를 확인한다.
- 테스트 중 다른 시스템 부하를 기록한다.
- 작은 입력만으로 확장성을 판단하지 않는다.
- 로그 출력 시간을 계산 시간과 섞지 않는다.

20. 테스트와 디버깅

20.1 불변식 중심 테스트

가능한 모든 실행 순서를 나열하려 하지 말고 항상 성립해야 하는 불변식을 검사한다.

- 전체 합계가 보존되는가?
- 큐 크기가 용량 범위를 벗어나지 않는가?
- 같은 작업이 두 번 완료되지 않는가?
- 종료 후 미처리 작업이 남지 않는가?

20.2 실행 순서 다양화

- 작업 수와 스레드 수를 바꾼다.
- 작은 임의 지연과 yield를 삽입한 테스트를 별도로 실행한다.
- 높은 반복 횟수로 스트레스 테스트한다.
- 단일 코어와 멀티코어 환경을 모두 시험한다.

테스트용 지연을 실제 동기화 수단으로 사용해서는 안 된다.

20.3 도구

- ThreadSanitizer: 데이터 경쟁 탐지
- AddressSanitizer: 메모리 오류 탐지
- 운영체제 프로파일러: CPU, 스케줄링, 캐시, 락 경합 분석
- 구조화된 로그와 trace ID: 작업 간 인과 관계 추적

로그 출력 자체가 락과 I/O를 사용하여 문제의 타이밍을 바꿀 수 있다는 점도 주의한다.

21. 흔한 오해

스레드가 많을수록 빠르다

아니다. CPU 중심 작업에서는 과도한 스레드가 문맥 교환과 캐시 손실을 증가시킨다.

atomic은 항상 mutex보다 빠르다

아니다. 경합이 심한 atomic도 캐시 라인을 계속 이동시키며, 복합 상태를 보호하려고 복잡한 lock-free 알고리즘을 만들면 오히려 느리고 위험할 수 있다.

서로 다른 변수를 쓰면 false sharing이 없다

아니다. 서로 다른 변수가 같은 캐시 라인에 있으면 성능 경합이 생길 수 있다.

비동기 코드는 자동으로 병렬 실행된다

아니다. 하나의 이벤트 루프에서 순차적으로 재개될 수 있다.

데이터 경쟁이 드물게만 발생하면 허용할 수 있다

아니다. C++의 데이터 경쟁은 undefined behavior다.

sleep으로 실행 순서를 맞출 수 있다

아니다. 시스템 부하와 스케줄러에 따라 언제든 실패할 수 있다.

22. 설계 체크리스트

- ☐ 동시성과 병렬성 중 실제 목표가 무엇인지 정의했는가?
- ☐ CPU 중심 작업과 I/O 중심 작업을 구분했는가?
- ☐ 순차 기준선과 기대 speedup을 계산했는가?
- ☐ 공유 상태와 소유권이 명확한가?
- ☐ 작업 사이의 의존성과 임계 경로를 파악했는가?
- ☐ 작업 크기가 스케줄링 비용보다 충분히 큰가?
- ☐ 스레드와 큐의 최대 크기가 제한되어 있는가?
- ☐ 유한 큐의 역압력 정책이 정의되어 있는가?
- ☐ mutex의 임계 영역이 불필요하게 크지 않은가?
- ☐ 조건 변수를 predicate와 함께 사용하는가?
- ☐ 여러 락의 획득 순서가 일관적인가?
- ☐ atomic의 메모리 순서를 설명할 수 있는가?
- ☐ 캐시 친화성과 false sharing을 측정했는가?
- ☐ 작업자별 부하 불균형을 관찰했는가?
- ☐ 블로킹 호출이 이벤트 루프를 멈추지 않는가?
- ☐ 취소와 종료 시 대기 중인 작업을 깨우는가?
- ☐ 결과뿐 아니라 불변식과 liveness도 테스트하는가?

23. 핵심 요약

1. 동시성은 여러 작업을 함께 진전시키는 구조이고 병렬성은 실제 동시 실행이다.
2. 비동기는 기다리지 않는 인터페이스이며 병렬성과 같은 뜻이 아니다.
3. 병렬화는 작업, 데이터, 파이프라인 또는 fork-join 방식으로 분해할 수 있다.
4. 단일 큐는 균형이 쉽지만 경합이 크고, 작업자별 큐는 확장성이 좋지만 불균형이 생길 수 있다.
5. work stealing은 유휴 작업자가 다른 큐의 작업을 가져와 불균형을 줄인다.
6. 멀티코어 성능은 연산 수뿐 아니라 캐시, NUMA와 메모리 대역폭에 좌우된다.
7. 동시성 정확성은 원자성, 순서, 가시성의 관점에서 검토한다.
8. C++ 데이터 경쟁은 undefined behavior이며 happens-before로 접근을 정렬해야 한다.
9. 복합 공유 상태에는 mutex, 단일 원자 상태에는 atomic을 우선 검토한다.
10. 이벤트 기반 구조에서는 블로킹 호출과 긴 CPU 작업을 분리해야 한다.
11. Amdahl의 법칙 때문에 작은 순차 부분도 전체 speedup을 제한한다.
12. 성능은 추측이 아니라 순차 기준선, 프로파일링과 반복 측정으로 판단한다.
13. 캐시 일관성은 메모리 순서와 프로그램의 동기화를 대신하지 않는다.

권장 학습 순서

TEXT

동시성·병렬성·비동기 구분

- 작업 분해 방식
- 실행 모델과 스케줄링
- 데이터 경쟁과 C++ 메모리 모델
- 동기화 도구
- 캐시·NUMA·부하 균등화
- 확장성 측정과 디버깅

관련 노트: 01.프로세스와 스레드 · OS 공부

[!success] 2단계 검증 기록

1차 문서 검증: 장 번호, frontmatter, Obsidian 링크, 표, 코드 블록을 검사했으며 핵심 C++20 예제를 MSVC `/std:c++20 /W4 /permissive-` 환경에서 컴파일하고 실행했다.

2차 PDF 검증: [ocs.pdf](#) 의 스레드와 병행성, 멀티코어 프로그래밍 과제, Amdahl의 법칙, 병렬성 유형, 동기화 도구·사례와 교착 상태 장을 다시 대조했다. [ostep.pdf](#) 의 멀티프로세서 스케줄링, 병행성 개요, 락, 병행 자료 구조, 조건 변수, 세마포어, 병행성 오류와 이벤트 기반 병행성 장을 다시 대조했다.

title: Mutex와 Condition Variable aliases:

tags:

created: 2026-08-02 updated: 2026-08-02

- 뮤텝스와 조건 변수
- Mutex and Condition Variable
- Lock and Condition Variable
- CS
- OS
- Cpp
- Concurrency
- Mutex
- ConditionVariable

Mutex와 Condition Variable

[!summary] 한 문장 정리

mutex는 공유 상태를 한 번에 한 스레드만 검사·변경하게 하고, **condition variable**은 그 mutex로 보호되는 predicate가 참이 될 때까지 스레드를 원자적으로 재우고 다시 깨우는 대기 도구다.

관련 노트: 00.메모리·객체 수명·RAII · 01.프로세스와 스레드 · 02.동시성과 병렬성

학습 목표

- data race, race condition, critical section을 구분한다.
- mutex가 보호하는 대상이 코드가 아니라 공유 상태와 불변식임을 이해한다.
- `lock_guard` , `unique_lock` , `scoped_lock` 을 목적에 맞게 사용한다.
- condition variable의 wait가 왜 mutex를 원자적으로 풀고 다시 획득하는지 설명한다.
- predicate, lost wakeup, spurious wakeup, Mesa semantics를 연결해서 이해한다.
- `notify_one` 과 `notify_all` 의 의미와 비용을 구분한다.
- 종료 가능한 bounded queue를 C++로 구현한다.
- deadlock, starvation, lock convoy, priority inversion을 구분하고 예방한다.

참고한 PDF 범위

- `C:\Users\user\Desktop\OS\cpp.pdf`
- PDF 744~759쪽: race condition, mutex, critical section, RAII lock, deadlock
- PDF 759~770쪽: producer-consumer, condition variable, `unique_lock` , `notify_one/all`
- `C:\Users\user\Desktop\OS\ostep.pdf`
- PDF 323~341쪽: lock의 요구 조건과 구현, spin과 sleep
- PDF 348~362쪽: condition variable, atomic wait, Mesa semantics, bounded buffer, spurious wakeup
- `C:\Users\user\Desktop\OS\ocs.pdf`
- PDF 230~255쪽 부근: critical-section problem, mutex lock, semaphore와 monitor
- PDF 249~250쪽: monitor의 condition, wait/signal 의미
- PDF 270~280쪽 부근: 운영체제·POSIX·Java의 동기화 사례

[!note] 자료 통합 원칙

PDF의 C/POSIX/Java 의사 코드는 동기화 원리를 확인하는 데 사용하고, 최종 예제는 현대 C++20의 RAII와 표준 라이브러리 계약에 맞춰 다시 작성한다. 특히 알림 횟수로 종료를 추론하지 않고, mutex로 보호되는 명시적인 `closed` 상태를 사용한다.

1. 문제의 출발점: 공유 상태

여러 스레드가 같은 메모리를 사용한다는 사실 자체는 문제가 아니다. 다음 조건이 겹치면 문제가 된다.

1. 둘 이상의 스레드가 같은 메모리 위치에 접근한다.
2. 그중 하나 이상이 쓰기다.
3. 충돌 접근 중 하나 이상이 비원자(non-atomic) 접근이다.
4. 접근 사이에 적절한 동기화가 없다.

C++에서 이런 충돌 접근은 **data race**이며 프로그램 전체가 undefined behavior가 된다.

CPP

```
int counter = 0;

void increment() {
    ++counter; // 여러 스레드가 동시에 실행하면 data race
}
```

`++counter` 는 소스 코드 한 줄이지만 반드시 하나의 원자적 기계 명령은 아니다. 개념적으로 읽기, 증가, 쓰기로 나뉠 수 있다.

TEXT

Thread A: read 0 — add 1 ————— write 1
 Thread B: ————— read 0 — add 1 — write 1
 결과: 두 번 증가했지만 1

1.1 Data race와 race condition

용어	의미
data race	C++ 메모리 모델에서 동기화되지 않은 충돌 접근. UB
race condition	실행 순서에 따라 결과의 정확성이 달라지는 더 넓은 논리 오류

mutex를 사용해 모든 메모리 접근의 data race를 없애도 “잔액 확인 후 송금” 같은 여러 단계의 논리가 서로 다른 임계 영역으로 찢어져 있으면 race condition이 남을 수 있다.

1.2 Critical section

critical section은 공유 불변식을 읽거나 변경하여 동시에 실행되면 안 되는 구간이다. 중요한 점은 **mutex가 코드 줄을 보호하는 것이 아니라 데이터와 불변식을 보호한다는 것이다.**

TEXT

mutex M이 보호하는 상태

```
queue, size, closed, 통계 카운터
불변식: size == queue.size()
불변식: closed면 새 push를 받지 않음
```

같은 상태를 어떤 경로에서는 mutex 없이 접근하거나 서로 다른 mutex로 보호하면 계약이 깨진다.

2. Mutex의 의미

mutex는 mutual exclusion의 줄임말이다. 한 스레드가 소유한 동안 다른 스레드의 획득을 막는다.

CPP

```
#include <mutex>

std::mutex mutex;
int counter = 0;

void increment() {
    std::lock_guard lock{mutex};
    ++counter;
}
```

2.1 세 가지 핵심 속성

- **상호 배제**: 임계 영역에는 한 번에 한 스레드만 진입한다.
- **소유권**: mutex를 성공적으로 lock한 스레드가 unlock해야 한다.
- **동기화**: 한 스레드의 unlock은 같은 mutex를 나중에 성공적으로 lock한 스레드와 happens-before 관계를 만든다.

마지막 속성 때문에 mutex는 단순한 “입장권”이 아니다. 임계 영역의 쓰기가 다음 소유자에게 올바르게 보이도록 메모리 가시성과 순서도 제공한다.

2.2 std::mutex 기본 연산

연산	의미
lock()	획득할 때까지 대기
try_lock()	즉시 시도하고 실패하면 <code>false</code>
unlock()	현재 스레드가 가진 소유권 해제

직접 `lock()` / `unlock()` 을 짚는 코드는 예외와 조기 반환에 취약하다.

CPP

```
mutex.lock();
update(); // 예외가 발생하면 unlock에 도달하지 못함
mutex.unlock();
```

따라서 RAII lock wrapper를 기본으로 사용한다.

3. RAII lock wrapper

3.1 std::lock_guard

범위 전체에서 하나의 mutex를 소유하면 가장 단순한 선택이다.

CPP

```
void update(State& state) {
    std::lock_guard lock{state.mutex};
    state.recalculate();
} // 자동 unlock
```

- 생성할 때 lock한다.
- 소멸할 때 unlock한다.
- 중간에 명시적으로 unlock하거나 다시 lock할 수 없다.
- 단순함이 장점이다.

3.2 std::unique_lock

소유권을 이동하거나 lock/unlock 시점을 바꿔야 할 때 사용한다.

CPP

```
std::unique_lock lock{mutex};
prepare_shared_state();
lock.unlock();
perform_slow_io();
```

`condition_variable::wait` 는 잠드는 동안 mutex를 풀고 깨어날 때 다시 획득해야 하므로 `unique_lock<mutex>` 를 받는다.

지원하는 대표 기능:

- deferred locking: `std::defer_lock`
- 이미 획득한 mutex 인수: `std::adopt_lock`
- 즉시 시도: `std::try_to_lock`
- `lock()` , `unlock()` , `owns_lock()`
- 이동 가능한 lock 소유권

`adopt_lock` 은 현재 스레드가 정말 mutex를 소유할 때만 사용한다. 계약이 틀리면 undefined behavior로 이어질 수 있다.

3.3 std::scoped_lock

여러 mutex를 함께 획득할 때 교착 회피 알고리즘을 적용한다.

CPP

```
void transfer(Account& from, Account& to, int amount) {
    if (&from == &to) {
        return;
    }

    std::scoped_lock lock{from.mutex, to.mutex};
    from.balance -= amount;
    to.balance += amount;
}
```

반대 순서로 두 mutex를 직접 획득하는 것보다 안전하다. 여러 코드 경로가 동일한 mutex 집합을 획득한다면 `scoped_lock` 또는 전역적으로 일관된 lock ordering을 사용한다.

3.4 lock wrapper 선택

필요	선택
범위 동안 mutex 하나를 단순 소유	<code>lock_guard</code>
condition variable 대기	<code>unique_lock</code>
중간 unlock/relock, 이동, 지연 획득	<code>unique_lock</code>
mutex 여러 개를 함께 획득	<code>scoped_lock</code>

4. Mutex는 OS에서 어떻게 기다리는가

표준은 구현 방식을 강제하지 않지만 일반적인 mutex는 두 경로를 조합한다.

TEXT

lock 요청

- └ 비경합 fast path: 원자 연산으로 사용자 공간에서 즉시 획득
- └ 경합 slow path:
 - 짧게 spin할 수 있음
 - 계속 실패하면 커널 대기 큐에 park/block
 - unlock 시 대기자 중 하나를 깨움

운영체제와 런타임은 futex 계열, 주소 기반 대기, 커널 객체 같은 기법을 사용할 수 있다. 정확한 구현은 플랫폼과 라이브러리에 따라 다르다.

4.1 Spin과 sleep

방식	장점	단점
spin	context switch 없이 곧바로 재시도	기다리는 동안 CPU 소비
sleep/block	CPU를 다른 작업에 양보	커널 진입·스케줄링·깨우기 비용

임계 영역이 매우 짧고 소유자가 다른 코어에서 곧 unlock할 가능성이 높으면 짧은 spin이 이득일 수 있다. 일반 애플리케이션은 `std::mutex` 구현에 맡기고 직접 spinlock을 만들지 않는 것이 좋다.

4.2 경합이 비싼 이유

- 캐시 라인 소유권 이동
- 원자적 read-modify-write
- 대기 큐 관리
- context switch와 scheduler 지연
- 깨어난 뒤 다시 mutex를 얻기 위한 경쟁

mutex가 있다는 이유보다 **경합하는 mutex**와 긴 임계 영역이 문제다.

5. Mutex만으로 부족한 문제

공유 큐가 비어 있을 때 소비자는 무엇을 해야 할까?

5.1 Busy waiting

```
CPP
for (;;) {
    std::lock_guard lock{mutex};
    if (!queue.empty()) {
        break;
    }
}
```

조건이 바뀌지 않았는데도 lock, 검사, unlock을 반복해 CPU와 캐시를 낭비한다.

5.2 Sleep polling

```
CPP
for (;;) {
    {
        std::lock_guard lock{mutex};
        if (!queue.empty()) {
            break;
        }
    }
    std::this_thread::sleep_for(std::chrono::milliseconds{10});
}
```

CPU 낭비는 줄지만 지연과 polling 주기 선택 문제가 남는다. 너무 짧으면 낭비하고 너무 길면 준비된 작업을 늦게 처리한다.

원하는 것은 "상태가 바뀔 때까지 재우고, 바뀌면 다시 검사"하는 기능이다. condition variable이 이 역할을 한다.

6. Condition Variable의 정체

condition variable은 조건 그 자체를 저장하지 않는다. **조건을 기다리는 스레드들의 대기 지점**이다.

CPP

```
std::mutex mutex;
std::condition_variable changed;
bool ready = false; // 실제 조건을 표현하는 공유 상태
```

여기서 기억해야 할 것은 다음이다.

- 상태: `ready`
- 상태 보호: `mutex`
- 상태 변경 알림: `changed`

notification은 상태가 아니다. 알림을 받은 횟수를 기록하거나 미래 waiter를 위해 보관하지 않는다.

6.1 기본 패턴

대기자:

CPP

```
std::unique_lock lock{mutex};
changed.wait(lock, [&] { return ready; });
use_ready_state(); // mutex를 다시 소유한 상태
```

변경자:

CPP

```
{
    std::lock_guard lock{mutex};
    ready = true;
}
changed.notify_one();
```

6.2 wait 의 핵심 원자 동작

대기자가 predicate가 거짓임을 확인한 뒤 mutex를 풀기 전에 변경자가 상태를 바꾸고 알림을 보내면 알림을 놓칠 수 있다. 이를 막기 위해 wait는 다음 전이를 하나의 대기 프로토콜로 수행한다.

TEXT

mutex를 소유한 대기자

- condition 대기 큐 등록과 mutex unlock을 원자적으로 연결
- 스레드 block
- notification 또는 spurious wakeup
- mutex를 다시 lock
- predicate 재검사

`wait` 에서 반환했다는 것은 notification을 받았다는 뜻보다 **mutex를 다시 소유한다**는 뜻이 중요하다. predicate가 참인지는 다시 확인해야 한다.

6.3 Predicate overload

CPP

```
changed.wait(lock, predicate);
```

개념적으로 다음과 같다.

CPP

```
while (!predicate()) {
    changed.wait(lock);
}
```

항상 predicate overload를 우선 사용하면 `while` 재검사 규칙을 코드에 내장할 수 있다.

7. 왜 `while` 인가

7.1 Mesa semantics

현대의 일반적인 condition variable은 notification을 받은 스레드가 즉시 실행되고 상태를 독점한다고 보장하지 않는다.

TEXT

Producer: 상태 변경 → notify Consumer A
 Consumer A: ready queue에서 대기
 Consumer B: 먼저 mutex 획득 → 상태 소비
 Consumer A: 나중에 mutex 획득 → 조건이 다시 거짓일 수 있음

notification은 “조건이 참이다”라는 증명서가 아니라 **상태가 변했을 가능성이 있다는 힌트**다. OSTEP은 이를 Mesa semantics로 설명한다.

7.2 Spurious wakeup

스레드는 대응하는 notification이 없어도 `wait` 에서 깨어날 수 있다. 표준 API가 허용하는 현상이므로 이유를 추측하지 말고 predicate를 재검사한다.

CPP

```
// 잘못된 패턴
if (!ready) {
    changed.wait(lock);
}
use_ready_state(); // 깨어난 뒤 ready가 거짓일 수 있음
```

CPP

```
// 올바른 패턴
changed.wait(lock, [&] { return ready; });
use_ready_state();
```

7.3 여러 waiter 사이의 경쟁

`notify_one` 이 어떤 waiter를 고르는지는 일반적으로 보장되지 않는다. 선택된 waiter가 원하는 세부 조건을 만족하지 못하거나, 깨어난 뒤 다른 스레드가 상태를 먼저 소비할 수 있다. predicate는 항상 최종 판단자다.

8. Lost Wakeup

condition variable의 가장 중요한 함정은 notification을 상태처럼 사용하는 것이다.

8.1 상태 없이 알림만 보내는 오류

CPP

```
// notifier
changed.notify_one(); // 현재 waiter가 없으면 효과가 사라짐

// 나중에 시작한 waiter
std::unique_lock lock{mutex};
changed.wait(lock); // 영원히 기다릴 수 있음
```

알림 이전에 waiter가 없으면 notification은 축적되지 않는다. 그래서 “일이 끝났다”, “큐에 항목이 있다”, “종료했다” 같은 사실을 공유 상태에 기록해야 한다.

8.2 검사와 sleep 사이의 경쟁

다음과 같이 조건 검사와 대기기를 mutex 없이 분리하면 lost wakeup이 생길 수 있다.

FORMULA · CALCULATION

Waiter: ready == false 확인
 Notifier: ready = true, notify
 Waiter: wait 시작
 결과: 앞으로 알릴 스레드가 없어 영원히 대기 가능

조건 검사, wait의 atomic unlock-and-sleep, 상태 변경을 같은 mutex 프로토콜 안에 넣어야 한다.

8.3 Condition variable has no memory

기억을 갖는 것은 predicate 상태다.

TEXT

잘못된 모델: `notify`가 토큰을 저장한다
 올바른 모델: `shared state`가 사실을 저장하고 `notify`는 재검사를 촉진한다

토큰 누적 자체가 필요하다면 semaphore, queue, future 같은 다른 추상화가 더 적합할 수 있다.

9. notify_one 과 notify_all

9.1 notify_one

대기자 중 하나를 깨우도록 시도한다.

적합한 경우:

- 항목 하나가 추가되어 소비자 하나만 진행할 수 있음
- 작업 하나를 worker 하나가 처리함
- 같은 predicate를 기다리는 waiter 중 하나면 충분함

선택되는 스레드의 공정성이나 순서는 기대하지 않는다.

9.2 notify_all

모든 대기자를 깨워 각각 predicate를 재검사하게 한다.

적합한 경우:

- `closed`, `shutdown`, `cancelled` 처럼 모든 waiter의 진행 조건이 바뀜
- 여러 waiter가 동시에 진행할 수 있는 상태 변화
- 서로 다른 세부 predicate를 하나의 condition variable에서 기다리는 covering condition

단점은 thundering herd다. 많은 스레드가 깨어나 mutex 하나를 놓고 경쟁한 뒤 대부분 다시 잠들 수 있다.

9.3 알림은 lock 안에서 해야 하는가

공유 상태 변경은 반드시 mutex 아래에서 수행한다. notification은 상황에 따라 lock 안이나 밖에서 호출할 수 있다.

```
CPP
{
    std::lock_guard lock{mutex};
    queue.push(value);
}
not_empty.notify_one();
```

먼저 unlock한 뒤 notify하면 깨어난 스레드가 즉시 mutex를 얻을 가능성이 커진다. 그러나 정확성은 "notify 위치" 요령이 아니라 다음 계약에 의존한다.

- predicate 상태는 같은 mutex로 보호한다.
- waiter는 mutex를 잡고 predicate를 검사한다.
- waiter는 predicate 기반 wait를 사용한다.

수명 관리나 특수한 프로토콜 때문에 lock 안에서 notify해야 하는 설계도 가능하다. 코드 전체의 불변식을 기준으로 판단한다.

10. condition_variable 과 unique_lock

`std::condition_variable`의 wait 함수는 `std::unique_lock<std::mutex>&`를 받는다.

CPP

```
std::unique_lock lock{mutex};
condition.wait(lock, predicate);
```

`lock_guard` 를 받지 않는 이유는 `wait`가 다음 동작을 해야 하기 때문이다.

1. 현재 mutex 소유 확인
2. 대기 등록과 함께 unlock
3. block
4. 깨어난 후 다시 lock
5. 소유권을 호출자에게 돌려줌

`unique_lock` 은 unlock/relock과 소유 상태 추적을 지원한다.

10.1 condition_variable_any

`condition_variable_any` 는 `unique_lock<mutex>` 외의 BasicLockable 형태도 사용할 수 있는 더 일반적인 도구다. 유연한 대신 구현 비용이 더 클 수 있다. 평범한 `std::mutex` 라면 `std::condition_variable` 을 기본으로 선택한다.

11. 종료 가능한 Bounded Queue

생산자-소비자 예제에서 종료를 정확히 표현하려면 알림 횟수나 처리 개수를 추측하지 말고 `closed` 상태를 둔다.

CPP · 1/2

```

#include <condition_variable>
#include <cstddef>
#include <deque>
#include <mutex>
#include <optional>
#include <stdexcept>
#include <type_traits>
#include <utility>

template <class T>
class BoundedQueue {
    static_assert(std::is_nothrow_move_constructible_v<T>,
                  "T must be nothrow move constructible");

public:
    explicit BoundedQueue(std::size_t capacity) : capacity_{capacity} {
        if (capacity == 0) {
            throw std::invalid_argument{"capacity must be positive"};
        }
    }

    BoundedQueue(const BoundedQueue&) = delete;
    BoundedQueue& operator=(const BoundedQueue&) = delete;

    bool push(T value) {
        std::unique_lock lock{mutex_};
        not_full_.wait(lock, [&] {
            return closed_ || queue_.size() < capacity_;
        });

        if (closed_) {
            return false;
        }

        queue_.push_back(std::move(value));
        lock.unlock();
        not_empty_.notify_one();
        return true;
    }

    std::optional<T> pop() {
        std::unique_lock lock{mutex_};
        not_empty_.wait(lock, [&] {
            return closed_ || !queue_.empty();
        });
    }

```

CPP · 2/2

```

    if (queue_.empty()) {
        return std::nullopt; // closed이며 남은 항목도 없음
    }

    T value = std::move(queue_.front());
    queue_.pop_front();
    lock.unlock();
    not_full_.notify_one();
    return value;
}

void close() {
    {
        std::lock_guard lock{mutex_};
        closed_ = true;
    }
    not_empty_.notify_all();
    not_full_.notify_all();
}

private:
    const std::size_t capacity_;
    std::mutex mutex_;
    std::condition_variable not_empty_;
    std::condition_variable not_full_;
    std::deque<T> queue_;
    bool closed_ = false;
};

```

11.1 보호되는 불변식

mutex가 보호하는 것은 다음 전체 상태다.

- queue_
- closed_
- queue_.size() <= capacity_
- 닫힌 뒤에는 새 항목을 받지 않는다는 규칙

11.2 왜 condition variable이 두 개인가

- not_empty_ : 소비자가 기다리는 조건
- not_full_ : 생산자가 기다리는 조건

하나의 condition variable로도 구현할 수 있지만, 소비자가 다른 소비자만 깨우거나 생산자가 다른 생산자만 깨우는 불필요한 wakeup이 늘 수 있다. OSTEP의 bounded-buffer 분석처럼 서로 다른 predicate 집합을 분리하면 의도가 명확하다.

11.3 왜 close() 는 둘 다 notify_all 하는가

종료는 빈 큐를 기다리는 모든 소비자와 가득 찬 큐를 기다리는 모든 생산자의 predicate를 바꾼다. 모두 깨어나 closed_ 를 확인해야 한다.

11.4 왜 느린 작업은 lock 밖에서 하는가

`pop()` 은 항목을 지역 변수로 이동하고 mutex를 풀 뒤 실제 처리를 맡긴다. I/O나 큰 계산을 임계 영역 안에서 하면 다른 생산자와 소비자가 모두 막힌다.

예제는 항목 이동 중 예외가 나서 queue 앞 원소가 부분적으로 변경되는 상황을 피하도록 `T`의 이동 생성자가 `noexcept` 일 것을 요구한다. 범용 라이브러리라면 복사 가능성, allocator, 예외 보장까지 타입 정책으로 설계해야 한다.

12. Bounded Queue 사용 예

CPP

```
#include <atomic>
#include <cassert>
#include <thread>
#include <vector>

int main() {
    BoundedQueue<int> queue{8};
    std::atomic<long long> sum{0};

    std::thread producer{[&] {
        for (int value = 1; value <= 1'000; ++value) {
            bool accepted = queue.push(value);
            assert(accepted);
        }
        queue.close();
    }};

    std::vector<std::thread> consumers;
    for (int i = 0; i < 4; ++i) {
        consumers.emplace_back([&] {
            while (auto value = queue.pop()) {
                sum.fetch_add(*value, std::memory_order_relaxed);
            }
        });
    }

    producer.join();
    for (auto& consumer : consumers) {
        consumer.join();
    }

    assert(sum.load() == 500'500);
}
```

queue의 내부 정확성은 mutex와 condition variable이 담당한다. 합계는 여러 소비자가 동시에 갱신하는 단일 값이므로 atomic을 사용했다. 복합 불변식과 대기 조건은 mutex가, 단순 누적 연산은 atomic이 담당하는 예다.

13. 시간 제한 대기

13.1 Predicate가 있는 `wait_for`

CPP

```
using namespace std::chrono_literals;

std::unique_lock lock{mutex};
if (!condition.wait_for(lock, 500ms, [&] { return ready; })) {
    // 500ms 안에 ready가 참이 되지 않음
    return Timeout{};
}
```

predicate overload는 spurious wakeup이 있어도 남은 시간을 관리하며 조건을 재검사한다.

13.2 직접 반복한다면 `absolute deadline`

상대 시간 `wait_for(500ms)` 를 루프마다 처음부터 다시 적용하면 spurious wakeup마다 전체 대기 시간이 늘어날 수 있다. 복잡한 반복 제어가 필요하면 한 번 계산한 deadline과 `wait_until` 을 사용한다.

CPP

```
auto deadline = std::chrono::steady_clock::now() + 500ms;
while (!ready) {
    if (condition.wait_until(lock, deadline) == std::cv_status::timeout) {
        if (!ready) {
            return Timeout{};
        }
    }
}
```

경과 시간 제한에는 시스템 시각 조정의 영향을 피하기 위해 `steady_clock` 을 우선한다.

14. Memory Ordering과 가시성

다음 패턴에서 consumer가 `ready == true` 를 보고도 오래된 `data` 를 읽는지 걱정할 수 있다.

CPP

```
// producer
{
    std::lock_guard lock{mutex};
    data = make_data();
    ready = true;
}
condition.notify_one();

// consumer
std::unique_lock lock{mutex};
condition.wait(lock, [&] { return ready; });
use(data);
```

동일한 mutex의 unlock과 뒤이은 lock이 synchronization을 형성하므로 producer의 임계 영역 쓰기는 consumer에게 보인다. notification 자체가 데이터를 publish하는 것이 아니라 **공유 mutex 프로토콜**이 가시성을 제공한다.

predicate가 atomic 변수여도 condition variable과 함께 사용할 상태라면 일반적으로 같은 mutex 아래에서 변경해 wait와 의 프로토콜을 명확하게 만드는 편이 안전하다. C++20의 `atomic::wait/notify` 처럼 atomic 전용 대기 모델을 선택 한다면 처음부터 그 계약으로 설계한다.

15. Deadlock

두 스레드가 서로 가진 자원을 기다리며 영원히 진행하지 못하는 상태다.

TEXT

```
Thread A: lock M1 — wait M2
Thread B: lock M2 — wait M1
```

15.1 Coffman의 네 조건

1. mutual exclusion
2. hold and wait
3. no preemption
4. circular wait

모두 성립하면 deadlock 가능성이 생긴다. 최소 하나를 구조적으로 제거한다.

15.2 예방 규칙

- 한 번에 하나의 mutex만 소유하도록 설계를 단순화한다.
- 여러 mutex가 필요하면 `scoped_lock` 으로 함께 획득한다.
- 시스템 전체에 일관된 lock ordering을 둔다.
- lock을 가진 채 외부 callback이나 알 수 없는 사용자 코드를 호출하지 않는다.
- lock을 가진 채 장시간 I/O, `join`, 다른 condition 대기를 하지 않는다.
- 임계 영역에서 다시 자신을 호출하는 구조를 피한다.

`recursive_mutex` 는 같은 스레드의 재귀 획득을 허용하지만 lock 구조와 불변식을 숨기기 쉽다. 재진입이 정말 타입의 계약인지 검토하고 일반 mutex와 함수 분리를 우선한다.

16. Deadlock 이외의 진행 문제

문제	의미
starvation	특정 스레드가 계속 선택되지 않아 진행하지 못함
livelock	스레드들이 상태를 계속 바꾸지만 유용한 진행이 없음
lock convoy	lock 소유권 전달과 scheduling 때문에 스레드 무리가 직렬 지연
priority inversion	낮은 우선순위 스레드가 가진 lock 때문에 높은 우선순위 스레드가 대기
thundering herd	notify_all 뒤 많은 waiter가 깨어나 한 자원을 두고 경쟁

`std::mutex` 나 condition variable이 강한 fairness를 보장한다고 가정하지 않는다. 실시간 요구가 있다면 운영체제의 priority inheritance와 실시간 동기화 primitive까지 검토해야 한다.

17. 흔한 잘못된 패턴

17.1 if 로 wait

```
CPP
if (queue.empty()) {
    condition.wait(lock);
}
```

Mesa semantics, 다른 consumer의 선점, spurious wakeup 때문에 틀릴 수 있다. predicate 기반 wait를 쓴다.

17.2 공유 상태 없이 notification만 사용

notification은 저장되지 않는다. 완료 여부나 queue 상태를 별도 predicate로 둔다.

17.3 predicate를 mutex 밖에서 읽음

```
CPP
while (!stopped) { // stopped가 같은 mutex나 atomic으로 보호되지 않으면 data race
    std::unique_lock lock{mutex};
    condition.wait(lock);
}
```

predicate 읽기와 쓰기를 같은 mutex 계약에 넣는다.

17.4 서로 다른 mutex로 같은 상태 보호

producer는 M1, consumer는 M2로 같은 queue를 보호하면 상호 배제가 성립하지 않는다. 상태마다 담당 mutex를 문서화한다.

17.5 lock을 가진 채 느린 작업

네트워크, 파일 I/O, 큰 계산, logging callback을 임계 영역 밖으로 옮긴다. 필요한 값을 지역 객체로 복사/이동한 뒤 unlock한다.

17.6 수동 unlock 누락

RAII wrapper를 사용한다. `unique_lock::unlock()` 이 필요해도 객체 소멸 시 남은 소유권은 안전하게 해제된다.

17.7 notify_all 로 논리 버그 숨기기

`notify_one` 에서는 멈추고 `notify_all` 에서만 동작한다면 waiter 종류와 predicate 설계를 다시 본다. 여러 조건을 하나의 CV에 섞었는지, 상태 변경이 누락됐는지 확인한다.

17.8 객체를 파괴하는 동안 waiter가 남아 있음

mutex와 condition variable은 사용하는 모든 스레드보다 오래 살아야 한다. 소유 객체를 파괴하기 전에 종료 상태 설정, `notify_all` , 스레드 join을 완료한다. 자세한 객체 수명 원칙은 00.메모리·객체 수명·RAII를 참고한다.

18. 어떤 동기화 도구를 선택할까

요구	우선 검토
복합 공유 상태와 불변식 보호	<code>mutex</code>
<code>mutex</code> 상태가 바뀔 때까지 대기	<code>condition_variable</code>
단일 값의 독립 원자 연산	<code>atomic<T></code>
누적 permit/토큰	<code>counting_semaphore</code>
한 번의 결과 전달	<code>promise/future</code>
여러 스레드의 일회성 합류	<code>latch</code>
반복 단계의 집결	<code>barrier</code>
읽기 다수, 쓰기 독점	<code>shared_mutex</code> 를 측정 후 검토

`condition variable`을 이벤트, 메시지 큐, 세마포어처럼 억지로 쓰지 않는다. 실제 원하는 상태 기계를 먼저 정의한다.

19. 성능 설계

19.1 임계 영역을 짧게

다음은 lock 안에 남겨야 한다.

- predicate 확인
- 공유 컨테이너의 최소 변경
- 불변식 갱신

다음은 보통 밖으로 옮긴다.

- I/O
- formatting과 logging
- 사용자 callback
- 큰 계산
- 반환할 데이터의 후처리

19.2 Lock granularity

큰 lock 하나는 설계가 단순하지만 병렬성이 줄 수 있다. 작은 lock 여러 개는 병렬성을 늘릴 수 있지만 deadlock, 메모리 비용, 복잡성이 커진다. 먼저 정확한 coarse-grained lock으로 구현하고 경합을 측정한 뒤 분할한다.

19.3 False sharing

서로 다른 mutex나 카운터라도 같은 cache line에 있으면 코어 사이 invalidation이 반복될 수 있다. 이것은 논리적 lock 경합과 다른 cache line 경합이다. 자세한 내용은 02.동시성과 병렬성을 참고한다.

20. 디버깅과 검증

20.1 높은 경고와 sanitizer

BASH

```
c++ -std=c++20 -Wall -Wextra -Wpedantic -g \
  -fsanitize=thread example.cpp
```

ThreadSanitizer는 data race와 일부 lock 오용을 찾는 데 유용하다. AddressSanitizer와 보통 별도 구성으로 실행한다. MSVC 환경에서는 플랫폼이 제공하는 concurrency debugger, Application Verifier, ETW 계열 도구도 검토할 수 있다.

20.2 스트레스 테스트

- 스레드 수를 코어 수보다 크게/작게 바꾼다.
- queue capacity를 1로 줄여 경합을 높인다.
- producer/consumer에 무작위 지연을 넣는다.
- 시작 직후 close, 가득 찬 상태에서 close, 빈 상태에서 close를 시험한다.
- notification 직전/직후에 yield하도록 변형한다.
- 수천 번 반복하고 timeout으로 hang을 감지한다.

20.3 로그의 한계

로그 추가는 scheduling을 바꿔 버그를 숨길 수 있다. timestamp만 보지 말고 thread id, lock 대상, 상태 전이, queue size를 구조화해서 기록한다. 로그 호출 자체를 공유 mutex 안에서 오래 수행하지 않는다.

21. 코드 리뷰 체크리스트

Mutex

- ☐ mutex가 보호하는 필드와 불변식이 명확한가?
- ☐ 같은 상태의 모든 접근이 같은 mutex 또는 명시된 atomic 계약을 따르는가?
- ☐ 수동 lock/unlock 대신 RAII를 사용하는가?
- ☐ 임계 영역 안에서 I/O, callback, join을 하지 않는가?
- ☐ 여러 mutex의 획득 순서가 일관적인가?
- ☐ mutex와 보호 대상의 수명이 모든 사용자보다 긴가?

Condition variable

- ☐ condition variable과 별도로 실제 predicate 상태가 있는가?
- ☐ predicate 읽기와 쓰기가 같은 mutex로 보호되는가?
- ☐ `wait(lock, predicate)` 를 사용하는가?
- ☐ 종료/취소 상태도 predicate에 포함되는가?

- ☐ 상태를 먼저 변경한 뒤 알림을 보내는가?
- ☐ `notify_one` 과 `notify_all` 의 선택 이유가 명확한가?
- ☐ 서로 다른 waiter 집합이면 CV를 분리할 필요가 없는가?
- ☐ timeout이 전체 deadline을 지키는가?

진행성과 성능

- ☐ deadlock cycle 가능성을 검토했는가?
- ☐ starvation과 fairness 요구가 있는가?
- ☐ lock 경합과 hold time을 측정했는가?
- ☐ `notify_all` 의 thundering herd가 허용되는가?
- ☐ shutdown 중 모든 waiter가 깨어나고 모든 thread가 join되는가?

22. 자주 나오는 질문

Q1. mutex가 있으면 condition variable은 왜 필요한가?

mutex는 현재 접근할 권리를 조정하지만 “조건이 나중에 참이 될 때까지 CPU를 양보하며 기다리기”를 직접 표현하지 않는다. condition variable은 mutex로 보호된 상태 변화를 효율적으로 기다리게 한다.

Q2. notify_one() 을 먼저 호출하고 나중에 wait해도 되는가?

notification 자체는 보관되지 않는다. 하지만 상태를 먼저 변경했다면 나중 waiter가 predicate를 확인하고 기다리지 않으므로 문제없다. 그래서 상태가 기억을 담당한다.

Q3. 깨운 스레드는 즉시 실행되는가?

아니다. ready 상태가 될 뿐이며 scheduler와 mutex 경쟁을 거쳐 나중에 실행될 수 있다. 그 사이 predicate가 바뀔 수 있어 재검사가 필요하다.

Q4. notify할 때 mutex를 꼭 잡아야 하는가?

공유 상태 변경은 mutex 아래에서 해야 한다. notify 자체는 올바른 프로토콜에서는 unlock 뒤에 할 수 있다. 단순 규칙보다 상태·수명·predicate의 전체 계약이 중요하다.

Q5. volatile 로 ready를 선언하면 되는가?

아니다. C++의 `volatile` 은 스레드 간 원자성이나 happens-before를 제공하지 않는다. mutex 또는 적절한 atomic을 사용한다.

Q6. recursive_mutex 가 deadlock을 해결하는가?

같은 스레드의 자기 재획득만 허용할 뿐 스레드 사이 lock cycle은 해결하지 않는다. 오히려 불분명한 재진입 구조를 숨길 수 있다.

Q7. condition variable이 FIFO로 깨우는가?

표준적으로 그런 공정성을 기대할 수 없다. 특정 순서가 정확성 요구라면 별도의 queue와 sequence 상태를 설계한다.

Q8. atomic이면 mutex보다 항상 빠른가?

아니다. atomic도 cache line 경쟁을 만들며 복합 불변식을 표현하기 어렵다. 올바른 추상화를 선택하고 실제 workload로 측정한다.

23. 최종 압축 정리

1. mutex는 공유 상태와 불변식에 대한 상호 배제와 메모리 가시성을 제공한다.
2. C++의 data race는 단순히 결과가 틀리는 문제가 아니라 undefined behavior다.
3. mutex 소유권은 `lock_guard` , `unique_lock` , `scoped_lock` 으로 RAII화한다.
4. condition variable은 조건을 저장하지 않으며 waiter를 재검사 시점까지 재운다.
5. 실제 기억은 mutex로 보호되는 predicate 상태가 담당한다.
6. wait는 대기 등록과 mutex unlock을 원자적으로 연결하고, 반환 전에 mutex를 재획득한다.
7. Mesa semantics와 spurious wakeup 때문에 predicate는 항상 반복해서 검사한다.
8. `wait(lock, predicate)` 를 기본 패턴으로 사용한다.
9. notification은 축적되지 않으므로 상태 없는 notify는 lost wakeup을 만들 수 있다.
10. 항목 하나에는 보통 `notify_one` , 종료처럼 전체 predicate가 바뀌면 `notify_all` 을 쓴다.
11. bounded queue는 `not_empty` , `not_full` , `closed` 상태로 설계하면 종료까지 명확해진다.
12. lock을 가진 채 느린 작업이나 알 수 없는 callback을 실행하지 않는다.
13. 여러 mutex는 `scoped_lock` 또는 일관된 순서로 획득한다.
14. 정확성을 먼저 확보하고 경합, hold time, wakeup 수를 측정해 최적화한다.

[!tip] 실전 기본형

공유 상태를 mutex 아래에서 바꾸고, `condition.wait(lock, predicate)` 로 기다리며, 상태 변경 후 필요한 범위만 `notify_one/all` 한다. 알림이 아니라 predicate가 진실의 원천이다.